

Implementation of a raytracer



Masterprojekt WS 12/13

Benedikt Mendorf

16. Oktober 2025

Betreuer: Prof. Dr.-Ing. Matthias Teschner
Computer Graphics
Albert-Ludwigs-Universität

Inhaltsverzeichnis

1	Einleitung	4
1.1	Zielsetzung	4
2	Homogene Koordinaten	5
2.1	Vektoren, Punkte und Normalen	5
2.2	Matrizen	6
3	Strahlverfolgung	8
3.1	Strahlgleichung	9
3.2	Projektionsebene	9
3.3	Orthografische Kamera	9
3.4	Lochkamera	10
4	Objekte und Schnittests	12
4.1	Ebene	13
4.2	Rechteck	13
4.3	Scheibe	14
4.4	Dreieck	14
4.5	Kugel	16
4.6	Selbstschnitt	17
5	Transformation	18
5.1	Grundlagen	18
5.2	Anwendung beim Raytracing	19
5.3	Transformation der Normalen	20
6	Schattierung	22
6.1	Die Rendergleichung	22
6.2	Matte Oberfläche	23
6.3	Spekulare Oberfläche - Phong	23
6.4	Spiegelnde Oberfläche	24
6.5	Dielektrikum	25
7	Triangulierte Objekte	26
7.1	Das OBJ-Format	26
7.2	Facettierte und glatte Schattierung	27
7.3	Instanziierung	28
8	Schnellere Schnittests durch Beschleunigungsstrukturen	29
8.1	Übersicht	29
8.2	Die „bounding interval hierarchy“	30
8.2.1	Konstruktion	30

Inhaltsverzeichnis

8.2.2	Traversierung	33
8.2.3	Geschwindigkeitsmessung	34
9	Texturierung	37
9.1	Normal Mapping	38
10	Bildkorrektur	41
10.1	Erzeugung geeigneter Abtastpunkte	41
11	Galerie	44

1 Einleitung

„Raytracing“ ist ein computergestütztes Verfahren, das versucht die reale Lichtausbreitung, sowie Licht-Objekt-Interaktion möglichst korrekt zu modellieren. In der Realität senden Lichtquellen Strahlen aus, die auf Objekte treffen, wodurch gewisse Farbanteile absorbiert und andere reflektiert werden. Gelangen diese reflektierte Strahlen auf einen Sensor, wie das menschliche Auge, kann anhand der unterschiedlichen Spektren ein Bild wahrgenommen werden. Beim „Raytracing“ geht man umgekehrt vor und sendet Strahlen vom Sensor aus in eine virtuelle Szene aus Objekten und Lichtern. Dadurch werden nur die relevanten Strahlen betrachtet, die im realen Fall vom Sensor wahrgenommen worden wären.

Aktuelle „Raytracer“ erzeugen bereits heute nahezu fotorealistische Bilder und werden z.B. zur Produktion von Kinofilmen oder Werbespots verwendet. Aufgrund der sehr komplexen Berechnungen eignet sich „raytracing“ im Gegensatz zur gerasterten Computergrafik eher selten für Echtzeitanwendungen.

1.1 Zielsetzung

Ziel des Projekts ist die eigenständige Implementierung eines „Raytracers“. Die dazu verwendete Programmiersprache ist C++. Sie zeichnet sich besonders durch ihre Performanz aus und ist damit hervorragend zur Bewältigung der großen Anzahl an benötigten Berechnungen geeignet.

Diese Ausarbeitung folgt dabei chronologisch den einzelnen Entwicklungsschritten. Zu Beginn werden die wichtigsten mathematischen Grundlagen geschaffen, die hauptsächlich in der linearen Algebra angesiedelt sind. In den folgenden Kapiteln wird dann erläutert wie Strahlen in eine Szene ausgesendet werden, die Definition einfacher Objekte aussieht und Strahl-Objekt Schnittpoints funktionieren, die zur Berechnung der Objekt-Schattierung benötigt werden. Darauf folgt die Einführung von Transformationen. Diese erlauben es die Gestalt von Objekten sowie deren Lage im Raum zu ändern. Im nächsten Schritt wird die Schattierung, d.h. die Oberflächeneigenschaften von Objekten berechnet. In diesem Zusammenhang spielen Rendergleichung, sowie Lichtquellen und Materialien eine wichtige Rolle.

Zu den aufwendigeren Methoden, die der „Raytracer“ im finalen Zustand beherrschen wird gehört neben der Visualisierung von transparenten Dielektrika wie etwa Glas, die Verwendung von komplexen triangulierten Objekten. Diese können eine enorme Anzahl an Schnittpoints erfordern, was die Implementierung einer Beschleunigungsstruktur zur Reduzierung der Schnittpoint notwendig macht.

Als Hauptleitfaden diente das Buch „Ray Tracing from the Ground Up“ von Kevin Suffern [Suf07].

2 Homogene Koordinaten

Der erste Schritt auf dem Weg zum funktionsfähigen „Raytracer“ besteht darin Objekte im dreidimensionalen Raum darstellen zu können. In der Computergrafik haben sich dafür die homogenen Koordinaten etabliert. Wie bei den inhomogenen Koordinaten existieren auch hier Spaltenvektoren. Jedoch bestehen diese aus vier anstatt drei Komponenten. Die vierte Komponente scheint zu Beginn unnütz, da sich die Position von Punkten und Objekten ebenso mit nur drei Komponenten eindeutig beschreiben lässt. Jedoch sorgt diese zusätzliche Komponente dafür, dass sich Transformationen auf eine einheitliche Art und Weise und zwar durch Multiplikation mit einer Transformationsmatrix durchführen lassen.

2.1 Vektoren, Punkte und Normalen

Neben der Vereinheitlichung von Transformationen bietet die zusätzliche Komponente, die ab jetzt mit w bezeichnet wird, den weiteren Vorteil, dass explizit zwischen Punkten und Vektoren unterschieden werden kann.

Nimmt w den Wert 0 an, so handelt es sich um einen Vektor. Vektoren dienen der reinen Richtungsangabe und bleiben z.B. anders als Punkte bei Translationen unverändert. Ein Punkt liegt vor, wenn w Werte ungleich 0 annimmt. Sie sind mathematisch gesehen Ortsvektoren und werden durch Translation verschoben. Punkte und Vektoren werden durch einen fett geschriebenen Kleinbuchstaben gekennzeichnet:

$$\text{Punkt: } \mathbf{p} = \begin{pmatrix} x \\ y \\ z \\ w \neq 0 \end{pmatrix} \quad \text{Vektor } \mathbf{v} = \begin{pmatrix} x \\ y \\ z \\ 0 \end{pmatrix}$$

Die Einträge x, y, z sind die Komponenten des Vektors oder Punktes und beziehen sich wie gewohnt auf die Koordinatenachsen eines dreidimensionalen kartesischen Koordinatensystems. Die Komponente w dient als Skalierungsfaktor. Die Länge eines Vektors $|\mathbf{v}|$ und Punktes $|\mathbf{p}|$ ist somit gegeben durch:

$$|\mathbf{v}| = \sqrt{x^2 + y^2 + z^2} \quad |\mathbf{p}| = \sqrt{\left(\frac{x}{w}\right)^2 + \left(\frac{y}{w}\right)^2 + \left(\frac{z}{w}\right)^2}$$

Wenn die Komponente eines Vektors oder Punktes gemeint ist, wird diese mittels Punkt-Satzzeichen dem fett geschriebenen Kleinbuchstaben angehängt. So ist mit $\mathbf{p}.x$ die x-Komponente des Punktes $\mathbf{p}$ gemeint.

In homogenen Koordinaten existiert ebenfalls ein Standardskalarprodukt sowie ein Kreuzprodukt, dessen Anwendung jedoch nur auf Vektoren sinnvoll ist:

$$\text{Skalarprodukt: } \mathbf{u} \cdot \mathbf{v} = \mathbf{u}.x \cdot \mathbf{v}.x + \mathbf{u}.y \cdot \mathbf{v}.y + \mathbf{u}.z \cdot \mathbf{v}.z + \mathbf{u}.w \cdot \mathbf{v}.w = \mathbf{u}^\top \mathbf{v} \quad (2.1)$$

$$\text{Kreuzprodukt: } \mathbf{u} \times \mathbf{v} = \begin{pmatrix} \mathbf{u}.y \cdot \mathbf{v}.z - \mathbf{u}.z \cdot \mathbf{v}.y \\ \mathbf{u}.z \cdot \mathbf{v}.x - \mathbf{u}.x \cdot \mathbf{v}.z \\ \mathbf{u}.x \cdot \mathbf{v}.y - \mathbf{u}.y \cdot \mathbf{v}.x \\ 0 \end{pmatrix} \quad (2.2)$$

wobei $\mathbf{u}.x, \mathbf{v}.x, \dots, \mathbf{v}.w$ die Komponenten der Vektoren $\mathbf{u}$ und $\mathbf{v}$ sind und $\mathbf{u}^\top$ der transponierte Vektor von $\mathbf{u}$ ist.

Vektoren, die senkrecht auf einer Oberfläche bzw. Tangentialebene eines Objekts stehen, werden in der Computergrafik zur besseren Unterscheidung als Normalen bezeichnet. Sie werden mit dem Buchstaben $\mathbf{n}$ gekennzeichnet und, anders als der Name suggeriert, nicht notwendigerweise normalisiert. Die Transformation von Normalen gestaltet sich ebenfalls anders als die von Vektoren, da Normalen nach einer Transformation weiterhin orthogonal zur Oberfläche ausgerichtet sein sollen.

2.2 Matrizen

In diesem Abschnitt werden die unterschiedlichen Transformationsmatrizen lediglich definiert. In Kapitel 5 wird im Detail gezeigt, wie Transformationen durchzuführen sind.

Matrizen werden beim „Raytracing“ hauptsächlich dazu verwendet, Objekte zu transformieren. Wegen der Verwendung von homogenen Koordinaten handelt es sich dabei um 4×4 -Matrizen. Sie werden durch einen fett geschriebenen Großbuchstaben gekennzeichnet.

Um Objekte entlang eines beliebigen Vektors $\mathbf{a}$ zu verschieben, verwendet man eine Translationsmatrix $\mathbf{T}$,

$$\mathbf{T}(\mathbf{a}) = \begin{bmatrix} 1 & 0 & 0 & a_x \\ 0 & 1 & 0 & a_y \\ 0 & 0 & 1 & a_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

wobei a_x, a_y, a_z die Komponenten des Vektors $\mathbf{a}$ sind.

Rotation um die Achsen des Koordinatensystems erreicht man durch die Rotationsmatrizen,

$$\begin{aligned} \mathbf{R}_x(\theta) &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, & \mathbf{R}_y(\theta) &= \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \\ \mathbf{R}_z(\theta) &= \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

2 Homogene Koordinaten

wobei jeweils um die x, y und z Achse um den Winkel θ (im Bogenmaß) im mathematisch positiven Sinn (also gegen den Uhrzeigersinn) gedreht wird.

Soll ein Punkt an den Achsen des zugrunde liegenden Koordinatensystems gespiegelt werden, so verwendet man Reflexionsmatrizen der Form,

$$\mathbf{S}_x = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{S}_y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{S}_z = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

wobei $\mathbf{S}_x$ an der x-Achse und die beiden anderen Matrizen entsprechen an der y-Achse und der z-Achse spiegeln.

Um Objekte entlang der Achsen auszudehnen oder zu kontrahieren verwendet man eine Skalierungsmatrix,

$$\mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

wobei s_x, s_y und s_z Faktoren sind, um die das Objekt entlang der Achsen skaliert wird.

3 Strahlverfolgung

Die rudimentäre Funktionsweise eines „Raytracers“ besteht darin, Strahlen in eine Szene, bestehend aus Objekten und Lichtquellen, auszusenden und zu überprüfen, ob diese ein Objekt treffen. Im Falle eines Schnittes wird die Schattierung des Objekts am Strahl-Objekt Schnittpunkt berechnet und diese Information wird entlang des Strahls auf eine Projektionsebene zurück propagiert. In Kapitel 6 wird die genaue Vorgehensweise zur Schattierungsberechnung erläutert.

In diesem Kapitel wird es darum gehen, die grundlegenden Komponenten vorzubereiten, die es dem „Raytracer“ ermöglichen, Strahlen in eine Szene auszusenden. Dazu benötigt man eine geeignete Repräsentation eines Strahls sowie ein Kameramodell, welches die Ansicht der Szene bestimmt. Die vom Kameramodell erzeugten Strahlen werden auch Primärstrahlen genannt. Im folgenden werden zwei Kameramodelle vorgestellt und zwar in Form einer orthografischen Kamera und in Form einer Lochkamera.

Die Besonderheit besteht bei der orthografischen Kamera darin, dass die Strahlen parallel zueinander verlaufen (s. Abb 3.1). Diese Art der Projektion wird häufig in technischen Zeichnungen verwendet, da Objekte nicht kleiner dargestellt werden, wenn sie sich von der Projektionsebenen entfernen, und somit ein besserer Vergleich der Größe von Objekten gegeben ist.

Die Lochkamera sorgt für eine perspektivische Projektion der Objekte auf die Projektionsebene. Dadurch erscheinen Objekte kleiner je weiter sie vom Beobachter entfernt sind (s. Abb 3.2).

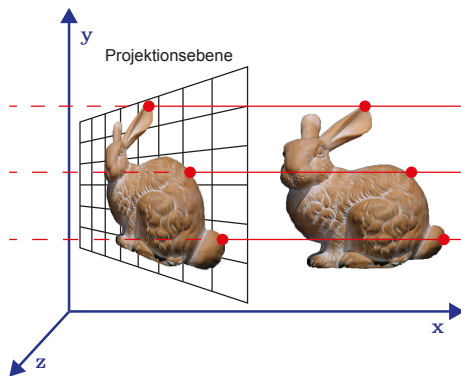


Abb. 3.1: Orthografische Kamera

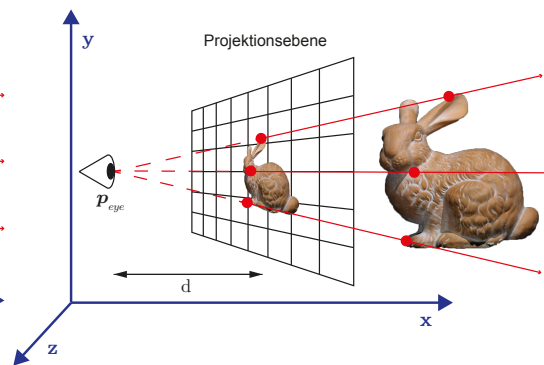


Abb. 3.2: Lochkamera

3.1 Strahlgleichung

Ein Strahl besitzt die folgende parametrisierte Form:

$$\mathbf{r} = \mathbf{o} + t \cdot \mathbf{d} \quad (3.1)$$

Der Punkt $\mathbf{o}$ ist der Ursprung des Strahls. Die Strahlrichtung ist durch den Vektor $\mathbf{d}$ gegeben. Der Parameter t kann beliebig gewählt werden und wird später dazu verwendet den Schnittpunkt $\mathbf{r}$ eines Strahls mit einem Objekt zu berechnen.

Um nun einen Strahl in eine Szene auszusenden, genügt es, seinen Ursprung $\mathbf{o}$ sowie Richtung $\mathbf{d}$ festzulegen. Dies ist die Aufgabe des Kameramodells.

3.2 Projektionsebene

Die Projektionsebene ist ein essentieller Bestandteil der nachfolgenden Kameramodelle. Tatsächlich handelt sich dabei nicht um eine Ebene, sondern um eine rechteckige Fläche mit der Höhe h und der Breite b . Sie ist in kleinere Projektionsebenen-Elemente (PEE) unterteilt, die nichts weiter als Quadrate gleicher Größe sind. PEE besitzen die Kantenlänge l_{PEE} . Es erweist sich als praktisch, die Größe bzw. das Seitenverhältnis der Projektionsebene festzulegen und anschließend anhand der Kantenlänge der PEE Einfluss auf die Auflösung des fertigen Bildes zu nehmen. Denn zum Ende der Bildsynthese enthalten die PEE Farbwerte, die auf Bildelemente eines Ausgabemediums übertragen werden können. Im Falle des Monitors besteht eine einfache Abbildung darin, den Farbwert eines PEE durch genau einen Bildpunkt des Monitors ausgeben zu lassen.

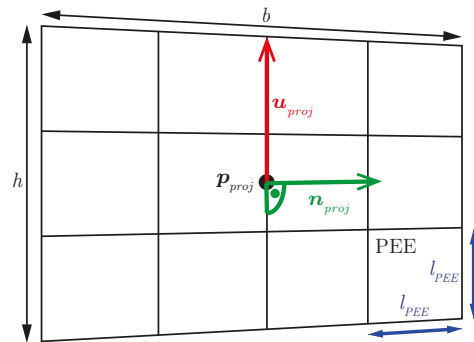


Abb. 3.3: Die Projektionsebene mit Höhe h , Breite b , Mittelpunkt $\mathbf{p}_{proj}$, „Up-Vektor“ $\mathbf{u}_{proj}$ und Normale $\mathbf{n}_{proj}$. Sie ist unterteilt in gleichgroße Quadrate (PEE) mit Kantenlänge l_{PEE} .

3.3 Orthografische Kamera

Das Prinzip der orthografischen Kamera ist recht simpel, daher eignet sie sich ausgezeichnet, um die korrekte Funktionsweise des „Raytracers“ zu Beginn der Implementationsphase zu überprüfen.

Sie besteht aus einer Projektionsebene (s. Abb. 3.3), die innerhalb des globalen Koordinatensystems eine Position $\mathbf{p}_{proj}$ und eine Ausrichtung besitzt, die durch die Normale $\mathbf{n}_{proj}$ und einem „Up-Vektor“ $\mathbf{u}_{proj}$ gegeben ist. Das Ziel besteht nun darin, Strahlen durch die einzelnen PEE hindurch in die Szene zu schicken. Trifft ein Strahl auf ein Objekt, wird am Schnittpunkt die Schattierung berechnet und diese Information dem PEE zugeordnet, welches vom entsprechenden Strahl durchdrungen wurde.

3 Strahlverfolgung

Zur Erzeugung der Strahlen empfiehlt es sich die Projektionsebene zu Beginn so zu positionieren, dass sich ihr Mittelpunkt im Ursprung des globalen Koordinatensystems befindet. Ihre Normale zeigt in die der z-Achse entgegengesetzte Richtung und der „Up-Vektor“ verläuft parallel zur y-Achse. Man kann nun relativ leicht den Mittelpunkt der einzelnen PEE berechnen. Dieser dient jeweils als Strahlursprung. Die Richtung der Strahlen verläuft parallel zur z-Achse und zwar in negativer Richtung. Die Position und Richtung der so erzeugten Strahlen werden mit $\mathbf{o}'$ und $\mathbf{d}'$ bezeichnet.

Diese Strahlen müssen nun vom globalen ins Kamera lokale Koordinatensystem transformiert werden. Dies gelingt, indem man eine Orthonormalbasis (ONB) aus $\mathbf{n}_{proj}$ und $\mathbf{u}_{proj}$ erzeugt, dann einen Basiswechsel durchführt und anschließend um den Vektor $\mathbf{v} = \mathbf{p}_{proj} - (0 \ 0 \ 0 \ 1)^T$ verschiebt.

Es sei vorausgesetzt, dass:

$$|\mathbf{n}_{proj}| = 1; \quad |\mathbf{u}_{proj}| = 1; \quad \mathbf{n}_{proj} \cdot \mathbf{u}_{proj} = 0$$

dann ist die ONB gegeben durch:

$$\mathbf{e}_1 = \mathbf{n}_{proj} \times \mathbf{u}_{proj}; \quad \mathbf{e}_2 = \mathbf{u}_{proj}; \quad \mathbf{e}_3 = -\mathbf{n}_{proj} \quad (3.2)$$

Basiswechsel durchführen mit:

$$\mathbf{o}'' = \mathbf{o}' \cdot x \cdot \mathbf{e}_1 + \mathbf{o}' \cdot y \cdot \mathbf{e}_2 + \mathbf{o}' \cdot z \cdot \mathbf{e}_3 + \mathbf{o}' \cdot w \cdot (0 \ 0 \ 0 \ 1)^T \quad (3.3)$$

$$\mathbf{d} = \mathbf{d}' \cdot x \cdot \mathbf{e}_1 + \mathbf{d}' \cdot y \cdot \mathbf{e}_2 + \mathbf{d}' \cdot z \cdot \mathbf{e}_3 \quad (3.4)$$

wobei $\mathbf{o}' \cdot x, \mathbf{d}' \cdot x, \dots, \mathbf{d}' \cdot w$ usw. die jeweilige Komponente von $\mathbf{o}'$ und $\mathbf{d}'$ bezeichnet. Anschließend um den Vektor $\mathbf{v}$ verschieben.

$$\mathbf{o} = \mathbf{o}'' + \mathbf{v}$$

Im Fall der orthografischen Kamera kann Gleichung 3.4 entfallen, da aufgrund des senkrechten Austrittswinkel der Strahlen aus der Projektionsebene einfach $\mathbf{d} = \mathbf{n}_{proj}$ gesetzt werden kann.

3.4 Lochkamera

Bei realen Lochkameras werden Objekte an der Lochblende um 180° gedreht auf einen Film oder Sensor abgebildet. Dieser Umstand lässt sich beim virtuellen Kameramodell gut vermeiden, indem man den Betrachter an die Position der Lochblende setzt und ihn auf die Szene richtet. Die Projektionsebene nimmt die Rolle des Films ein und wird zwischen Betrachter und Szene eingebracht. Je kleiner das Loch ist, desto schärfer wird das Bild in der Realität, da störende Randstrahlen abgeschirmt werden.

Die Position des Betrachters $\mathbf{p}_{eye}$ dient als Startpunkt für die Strahlen und kann als unendlich kleines Loch angesehen werden, wodurch maximal scharfe Bilder erzeugt werden. Bei

3 Strahlverfolgung

der orthografischen Kamera spielt es generell keine Rolle, ob der Startpunkt der Strahlen auf der Projektionsebenen liegt oder irgendwo davor. Bei der Lochkamera verhält es sich komplett anders. Die Entfernung des Betrachters, in Abb. 3.2 mit d gekennzeichnet, entscheidet nämlich darüber, wie groß das Sichtfeld ausfällt. Durch folgende Formel lässt sich d bestimmen:

$$d = \frac{b}{2 \cdot \tan(\phi_{hfov} \cdot \frac{1}{2})}$$

b ist die Breite der Projektionsebene. ϕ_{hfov} ist der Winkel, der zwischen $\mathbf{p}_{eye}$ und Mittelpunkt der Projektionsebene, sowie $\mathbf{p}_{eye}$ und Mittelpunkt der seitlichen Kante der Projektionsebene aufgespannt wird. Das Sichtfeld des Menschen beträgt fast 180° , was im Bogenmaß einem ϕ_{hfov} von π entsprechen würde. Da der Monitor jedoch nur einen Teil des menschlichen Sichtfelds einnimmt, hat sich ein Wert von $\frac{\pi}{2}$ für ϕ_{hfov} als geeignet herausgestellt.

Um Strahlen zu erzeugen, richtet man die Projektionsebene wie bei der orthografischen Kamera gezeigt aus, d.h. im Mittelpunkt des globalen Koordinatensystems und die Normale in Richtung negativer z-Achse. Der Betrachter wird an Position $\mathbf{p}_{eye} = (0 \ 0 \ d \ 1)^\top$ gesetzt. Sei $\mathbf{m}_{PEE}$ der Mittelpunkt eines PEE, dann ergibt sich die vorläufige Richtung $\mathbf{d}'$ eines Strahls aus $\mathbf{d}' = \mathbf{m}_{PEE} - \mathbf{p}_{eye}$. Anschließend erzeugt man eine ONB nach Gleichung 3.2 und berechnet die endgültige Strahlrichtung $\mathbf{d}$, wie in 3.4 gezeigt. Für den Strahlursprung lässt sich einfach $\mathbf{o} = \mathbf{p}_{eye}$ setzen.

4 Objekte und Schnitttests

Bis jetzt ist der „Raytracer“ in der Lage Strahlen gemäß eines gewünschten Kameramodells in die Szene zu schicken. Der nächste Schritt besteht nun darin, die Szene mit Objekten zu bevölkern. Anschließend muss überprüft werden, ob ein ausgesandter Strahl eines der Objekte trifft. Im Falle, dass mehrere Objekte hintereinander liegen und es somit zu mehreren Schnitten kommt, wird immer der Schnittpunkt gesucht, der dem Betrachter am nächsten ist. Abb. 4.1 zeigt einen solchen Fall. Die Durchdringung der Objekte, wird durch Verwendung des zum Betrachter nächsten Schnittpunkts korrekt dargestellt.

Neben dem nächstgelegenen Schnittpunkt wird auch noch eine Normale benötigt, um die Schattierung des Objekts berechnen zu können. Das folgende Kapitel zeigt die Definition verschiedener Objekte und demonstriert den Ablauf der jeweiligen Schnitttests. Zudem wird für gefundene Schnittpunkte eine Normale erzeugt, die zur Schattierungsberechnung benötigt wird.

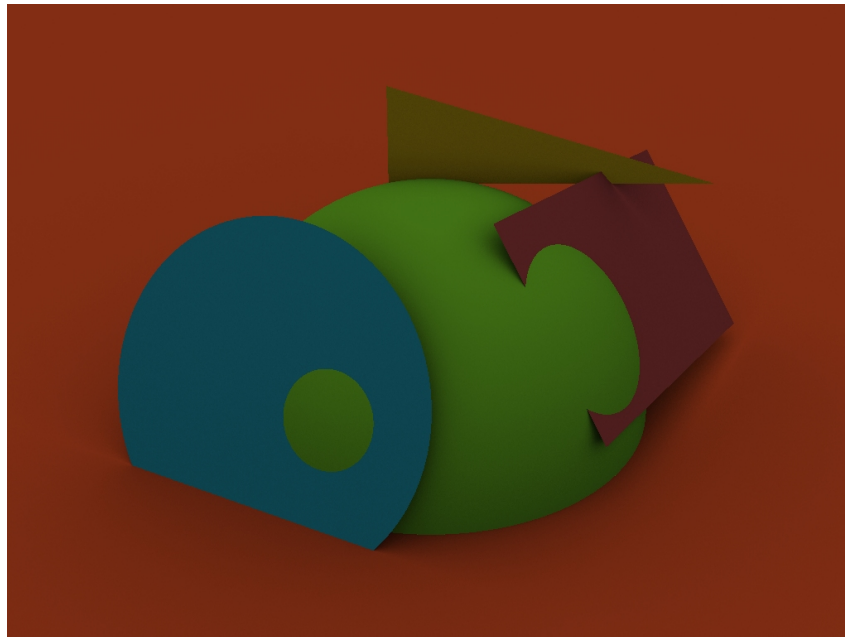


Abb. 4.1: Schnitt einer Kugel mit Ebene, Scheibe, Rechteck und Dreieck. Der zum Betrachter nächste Schnittpunkt wird zu korrekten Darstellung der Objektdurchdringungen benötigt.

4.1 Ebene

Der Schnitttest mit einer Ebene gestaltet sich sehr einfach und dient als Grundlage für weitere Schnitttests mit Objekten wie Rechtecken, Dreiecken oder Scheiben. Dazu betrachtet man folgende implizite Form der Ebene:

$$(\mathbf{p} - \mathbf{a}) \cdot \mathbf{n} = 0 \quad (4.1)$$

wobei $\mathbf{a}$ ein Ortsvektor in der Ebene ist und $\mathbf{n}$ eine Normale, die somit senkrecht auf der Ebene steht. Den impliziten Charakter erhält die Gleichung dadurch, dass sie eine Bedingung an Punkte $\mathbf{p}$ in der Ebene stellt. Ein Punkt liegt nämlich genau dann in der Ebene, wenn die Normale $\mathbf{n}$ senkrecht auf dem Vektor von $\mathbf{a}$ nach $\mathbf{p}$ steht und das Skalarprodukt somit gleich Null ist.

Das Ziel ist es jetzt, die Strahlgleichung 3.1 einzusetzen und den Parameter t zu bestimmen, wobei nur für positive t tatsächlich ein Schnitt vorliegt.

$$\begin{aligned} 0 &= (\mathbf{o} + t \cdot \mathbf{d} - \mathbf{a}) \cdot \mathbf{n} \\ \text{Finale Form: } t &= \frac{(\mathbf{a} - \mathbf{o}) \cdot \mathbf{n}}{\mathbf{d} \cdot \mathbf{n}} \end{aligned} \quad (4.2)$$

Zum Schattieren der Ebene wird die Normale $\mathbf{n}$ verwendet, die bereits zur Definition der Ebene diente. Der Schnittpunkt $\mathbf{r}$ lässt sich nun leicht durch Einsetzen von t in die Strahlgleichung 3.1 bestimmen, da Strahlursprung $\mathbf{o}$ und Strahlrichtung $\mathbf{d}$ durch das Kameramodell bereits festgelegt sind.

4.2 Rechteck

Eine einfache Repräsentation eines Rechtecks besteht darin, den linken unteren Punkt $\mathbf{a}$ anzugeben und die Ausdehnung des Rechtecks durch zwei orthogonal verlaufende Vektoren $\mathbf{e}_1$ und $\mathbf{e}_2$ festzulegen (siehe Abbildung 4.2). Diese Informationen sind bereits ausreichend, um die auf dem Rechteck senkrecht stehende Normale $\mathbf{n}$ durch den Term $(\mathbf{e}_1 \times \mathbf{e}_2) \cdot (\mathbf{e}_1 \times \mathbf{e}_2)^{-1}$ zu bestimmen. Dennoch ist es in der Implementation sinnvoll, die explizite Angabe der Normalen zu erlauben, da auf diese Weise festgelegt werden kann, für welche Seite die Schattierungsberechnungen durchgeführt werden soll.

Der eigentliche Schnitttest verläuft nun in zwei Schritten. Zuerst wird ein Ebenen-Schnitttest nach Gleichung 4.2 durchgeführt, wodurch man den Parameter t erhält. Durch die

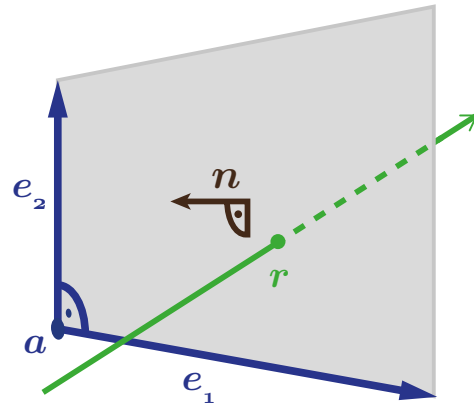


Abb. 4.2: Definition eines Rechtecks durch linken unteren Stützpunkt $\mathbf{a}$, Kantenvektoren $\mathbf{e}_1, \mathbf{e}_2$ und Normale $\mathbf{n}$. Ein Strahl schneidet das Rechteck am Schnittpunkt $\mathbf{r}$.

Strahlgleichung 3.1 lässt sich nun, falls existent, exakt der Schnittpunkt $\mathbf{r}$ berechnen.

Im nächsten Schritt erzeugt man einen Vektor $\mathbf{v} = \mathbf{r} - \mathbf{a}$. Diesen projiziert man auf Basisvektoren, die die gleiche Richtung wie $\mathbf{e}_1$ und $\mathbf{e}_2$ besitzen. Damit erhält man die Länge des Vektors $\mathbf{v}$ in Richtung von $\mathbf{e}_1$ und $\mathbf{e}_2$. Die so projizierten Vektoren $\mathbf{v}_{\mathbf{e}_1}$ und $\mathbf{v}_{\mathbf{e}_2}$ erhält man daher durch:

$$\mathbf{v}_{\mathbf{e}_1} = \frac{\mathbf{v} \cdot \mathbf{e}_1}{|\mathbf{e}_1|} \quad \mathbf{v}_{\mathbf{e}_2} = \frac{\mathbf{v} \cdot \mathbf{e}_2}{|\mathbf{e}_2|}$$

Der Punkt $\mathbf{r}$ ist genau dann ein Schnittpunkt des Rechtecks, wenn die Längen von $\mathbf{v}_{\mathbf{e}_1}$ und $\mathbf{v}_{\mathbf{e}_2}$ positiv sind und nicht länger als ihre jeweiligen Projektionsachsen. Daher müssen folgende Ungleichungen gelten:

$$0 \leq |\mathbf{v}_{\mathbf{e}_1}| \leq |\mathbf{e}_1| \quad \text{und} \quad 0 \leq |\mathbf{v}_{\mathbf{e}_2}| \leq |\mathbf{e}_2|$$

Momentan muss noch eine Quadratwurzel berechnet werden, um $|\mathbf{v}_{\mathbf{e}_1}|$ zu erhalten. Diese Operation ist jedoch relativ teuer und lässt sich einfach vermeiden, indem man die Ungleichung umformt:

$$\text{Final:} \quad 0 \leq \mathbf{v} \cdot \mathbf{e}_1 \leq |\mathbf{e}_1|^2 \quad \text{und} \quad 0 \leq \mathbf{v} \cdot \mathbf{e}_2 \leq |\mathbf{e}_2|^2 \quad (4.3)$$

4.3 Scheibe

Eine Scheibe wird durch drei Komponenten definiert. Dem Mittelpunkt $\mathbf{a}$, einer Normalen $\mathbf{n}$, die senkrecht auf der Scheibe steht und somit deren Lage festlegt, und dem Radius r . Zuerst wird auch hier ein Ebenen-Schnitttest nach Gleichung 4.2 durchgeführt. Anschließend wird geprüft, ob der Schnittpunkt $\mathbf{r}$ (berechnet durch Gleichung 3.1) in der Scheibe liegt. Der Ebenenschnittpunkt $\mathbf{r}$ liegt in der Scheibe, wenn folgende Ungleichung erfüllt ist:

$$0 \leq |\mathbf{r} - \mathbf{a}|^2 \leq r^2 \quad (4.4)$$

4.4 Dreieck

Der nun folgende Schnitttest wird im Buch „Physically Based Rendering“ [Mat10] beschrieben und geht zurück auf die Arbeit von Thomas Möller und Ben Trumbore [MT97]. Wie bei den bisher vorgestellten Schnitttests könnte man zuerst den Schnittpunkt mit der durch das Dreieck aufgespannten Ebenen berechnen und anschließend durch einen Halbseitentest feststellen, ob dieser Schnittpunkt innerhalb des Dreiecks liegt. Die Methode von [MT97] löst dieses Problem allerdings eleganter durch die Verwendung von baryzentrischen Koordinaten, die auf die Arbeit von August Ferdinand Möbius [Mö27] zurückgehen. Man erhält dadurch nicht nur die Position des Schnittpunktes im globalen Koordinatensystem, sondern auch dessen Lage innerhalb des Dreiecks, ausgedrückt durch baryzentrische Koordinaten. Letzterer Aspekt wird noch eine wichtige Rolle beim Texturieren von Objekten in Kapitel 9 spielen. Zudem wird in [MT97] gezeigt, dass sich der Speicherverbrauch durch das Vermeiden des Strahl-Ebenen Schnitts verringern lässt.

4 Objekte und Schnitttests

Ein Dreieck ist gegeben durch drei Punkte $\mathbf{p}_0, \mathbf{p}_1$ und $\mathbf{p}_2$. Die Normale $\mathbf{n}$ lässt sich aus dem Kreuzprodukt $\mathbf{n} = (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)$ errechnen. Das Ziel ist es nun, einen Schnitt festzustellen und den Schnittpunkt $\mathbf{r}$ durch baryzentrische Koordinaten auszudrücken. Dazu werden die Koeffizienten α, β und γ eingeführt, die als Gewichte für die Punkte $\mathbf{p}_0, \mathbf{p}_1$ und $\mathbf{p}_2$ dienen. Sie besitzen daher die gleiche Bedeutung wie die Massen bei der Bestimmung des Massenschwerpunkts. Der gesuchte Schnittpunkt $\mathbf{r}$ lässt sich somit als gewichtetes Mittel der Dreieckspunkte darstellen:

$$\mathbf{r} = \alpha \cdot \mathbf{p}_0 + \beta \cdot \mathbf{p}_1 + \gamma \cdot \mathbf{p}_2 \quad (4.5)$$

Da nur Schnittpunkte gesucht sind, die innerhalb des Dreiecks liegen, müssen die Gewichte stets positiv sein. Außerdem gilt für jeden Punkt innerhalb des Dreiecks, dass sich die Gewichte zu 1 summieren. Es gelten somit die folgenden Bedingungen:

$$\text{Bedingung 1:} \quad 0 \leq \alpha \leq 1; \quad 0 \leq \beta \leq 1; \quad 0 \leq \gamma \leq 1 \quad (4.6)$$

$$\text{Bedingung 2:} \quad \alpha + \beta + \gamma = 1 \quad (4.7)$$

Die zweite Bedingung 4.7 ermöglicht es α durch β und γ auszudrücken. Nach Einsetzen der Strahlgleichung 3.1 für $\mathbf{r}$ ergibt sich folgende Form:

$$\mathbf{o} + t \cdot \mathbf{d} = (1 - \beta - \gamma) \cdot \mathbf{p}_0 + \beta \cdot \mathbf{p}_1 + \gamma \cdot \mathbf{p}_2$$

Der nächste Schritt besteht darin, diese Gleichung in eine Form zu bringen, die mit Hilfe der Cramerschen Regel lösbar ist. Sie hat die Form $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, wobei eine Lösung für $\mathbf{x}$ gesucht ist. Nach dem Umformen erhält man:

$$\underbrace{\begin{bmatrix} \mathbf{d} & \mathbf{p}_1 - \mathbf{p}_0 & \mathbf{p}_2 - \mathbf{p}_0 \end{bmatrix}}_{\mathbf{A}} \cdot \underbrace{\begin{pmatrix} t \\ \beta \\ \gamma \end{pmatrix}}_{\mathbf{x}} = \underbrace{\mathbf{o} - \mathbf{p}_0}_{\mathbf{b}}$$

Abkürzend wird hier nur die resultierende Gleichung zitiert. Die vollständige Herleitung kann in [Mat10] (S. 140 ff) nachgelesen werden. Mit folgenden Substitutionen

$$\begin{aligned} \mathbf{e}_1 &= \mathbf{p}_1 - \mathbf{p}_0; & \mathbf{e}_2 &= \mathbf{p}_2 - \mathbf{p}_0; & \mathbf{s} &= \mathbf{o} - \mathbf{p}_0; \\ \mathbf{s}_1 &= \mathbf{d} \times \mathbf{e}_2; & \mathbf{s}_2 &= \mathbf{s} \times \mathbf{e}_1; \end{aligned}$$

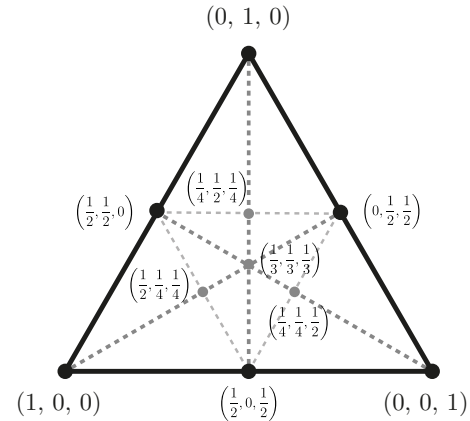


Abb. 4.3: Baryzentrische Koordinaten im Dreieck. Alle Punkte im Dreieck können als gewichtete Summe der 3 Eckpunkten dargestellt werden.

4 Objekte und Schnittpunkte

sind nun alle Komponenten bekannt, um die finale Gleichung 4.8 zu lösen.

$$\text{Finale Form: } \begin{pmatrix} t \\ \beta \\ \gamma \end{pmatrix} = \frac{1}{s_1 \cdot e_1} \cdot \begin{pmatrix} s_2 \cdot e_2 \\ s_1 \cdot s \\ s_2 \cdot d \end{pmatrix} \quad (4.8)$$

Abschließend ist zu bemerken, dass die Berechnung der Gleichung 4.8 unter Umständen frühzeitig beendet werden kann, falls kein Schnitt vorliegt. Dazu verwendet man Bedingung 4.6 und 4.7 sowie die Tatsache, dass eine Division durch 0 nicht definiert ist. Algorithmus 1 veranschaulicht in welcher Reihenfolge deshalb die einzelnen Komponenten berechnet werden sollten und welche Bedingungen verletzt sein müssen, damit die Berechnungen frühzeitig mit dem Rückgabewert „false“ beendet werden.

Algorithmus 1 Dreieck Schnittpunkt

```

1: divisor ←  $s_1 \cdot e_1$ 
2: if divisor = 0 then
3:   return false
4:  $\beta \leftarrow s_1 \cdot s \cdot \textit{divisor}^{-1}$ 
5: if  $\beta < 0$  or  $\beta > 1$  then
6:   return false
7:  $\gamma \leftarrow s_2 \cdot d \cdot \textit{divisor}^{-1}$ 
8: if  $\gamma < 0$  or  $\beta + \gamma > 1$  then
9:   return false
10:  $t \leftarrow s_2 \cdot e_2 \cdot \textit{divisor}^{-1}$ 
11: return true

```

4.5 Kugel

Eine Kugel besitzt einen Mittelpunkt c und einen Radius r . An alle Punkte p auf der Kugeloberfläche wird durch die implizite Form folgende Bedingung gestellt:

$$(p - c) \cdot (p - c) - r^2 = 0$$

D.h. die Entfernung eines Punktes p vom Mittelpunkt c muss genau dem Radius entsprechen. Nach Einsetzen der Strahlgleichung 3.1 in obige Gleichung und Umformen erhält man eine quadratische Gleichung, die z.B. durch die a-b-c-Formel gelöst werden kann:

$$\begin{aligned} & (o + td - c) \cdot (o + td - c) - r^2 = 0 \\ & \underbrace{d \cdot d}_a t^2 + \underbrace{2(o - c) \cdot d}_b t + \underbrace{(o - c) \cdot (o - c) - r^2}_c = 0 \end{aligned}$$

Eine quadratische Gleichung hat typischerweise eine, zwei oder keine Lösung. Dies hängt davon ab, wie der Strahl die Kugel trifft und welche Werte die Diskriminante annimmt. Verfehlt der Strahl die Kugel, gibt es keine Lösung, tangiert er sie, gibt es eine Lösung und

wenn er sie durchdringt, gibt es einen Ein- und Austrittspunkt und somit zwei Lösungen. Ein Strahl kann auch innerhalb einer Kugel beginnen, wobei mathematisch zwei Lösungen existieren, jedoch befindet sich der Schnittpunkt einer der Lösungen hinter dem Strahlursprung und ist unbrauchbar. Dies wird deutlich durch den Parameter t , der in diesem Fall einen negativen Wert annimmt. Ähnlich ist der Fall, in dem der Strahl exakt auf der Kugeloberfläche beginnt. Es ist somit sinnvoll, einen gefundenen Schnittpunkt nur dann als solchen zuzulassen, wenn $t > 0$ gilt.

Algorithmus 2 demonstriert die Vorgehensweise und gibt **true** zurück bei erfolgreichem Schnitt:

Algorithmus 2 Kugel Schnitttest

```

1: diskriminante  $\leftarrow b \cdot b - 4 \cdot a \cdot c$ 
2: if diskriminante  $< 0$  then
3:   return false
4:  $t \leftarrow -b - \sqrt{\textit{diskriminante}} \cdot (2a)^{-1}$  ▷ Teste zuerst die kleinere Wurzel..
5: if  $t > 0$  then
6:   return true
7:  $t \leftarrow -b + \sqrt{\textit{diskriminante}} \cdot (2a)^{-1}$  ▷ .. jetzt die größere Wurzel
8: if  $t > 0$  then
9:   return true
10: return false

```

Nun kann man wieder t in Strahlgleichung 3.1 einsetzen und Schnittpunkt $\mathbf{r}$ berechnen. Zuletzt fehlt nur noch die Normale am Schnittpunkt. Diese ergibt sich einfach aus:

$$\mathbf{n} = \frac{\mathbf{r} - \mathbf{c}}{|\mathbf{r} - \mathbf{c}|}$$

4.6 Selbstschnitt

Bei den oben beschriebenen mathematischen Herleitungen der Schnitttests bleibt ein Problem verborgen, das erst in der Implementation erkennbar wird. Dabei handelt es sich um den Selbstschnitt. Ein Strahl wird von der Oberfläche eines Objekts ausgesandt und der Schnitttest erkennt fälschlicherweise einen Schnitt mit eben diesem Objekt. Aufgrund von Ungenauigkeiten bei der Berechnung und Darstellung von Fließkommazahlen liefert der Test $t > 0$ oft noch falsche Schnitte, weshalb es sich bewährt hat, gegen einen sehr kleinen Wert k_ϵ zu testen, also $t > k_\epsilon$, wobei sich ein Wert von 10^{-4} für k_ϵ bewährt hat.

5 Transformation

In Abschnitt 2.2 wurden bereits Transformationsmatrizen eingeführt, die vorab verschiedene Transformationsarten vorstellten. In diesem Kapitel wird es darum gehen, zu zeigen, wie die Transformation für Objekte genau durchzuführen ist. Anders als bei der gerasterten 3D-Computergrafik bestehen Objekte beim „Raytracing“ nicht notwendigerweise aus einer Menge von Polygonen, deren Punkte sich relativ einfach transformieren lassen. So existieren z.B. Objekte mit komplexeren impliziten Darstellungsformen als die bereits vorgestellten. Für diese Objekte erscheint eine direkte Transformation aussichtslos. Der Trick wird deshalb darin bestehen, nicht direkt die Objekte, sondern den Strahl auf eine geeignete Art und Weise zu transformieren, um so zum selben Ergebnis zu kommen.

5.1 Grundlagen

Die Transformation von Punkten oder Vektoren funktioniert über das Matrix-Vektor-Produkt:

$$\mathbf{p}' = \mathbf{T} \cdot \mathbf{p}$$

mit $\mathbf{T} \in \mathbb{R}^{4 \times 4}$ und $\mathbf{p} \in \mathbb{R}^4$. Punkt $\mathbf{p}'$ geht aus der Transformation von $\mathbf{p}$ mit Transformationsmatrix $\mathbf{T}$ hervor.

Transformationen lassen sich durch linksseitige Multiplikation mit der inversen Transformationsmatrix $\mathbf{T}^{-1}$ umkehren:

$$\begin{aligned} \mathbf{T}^{-1} \cdot \mathbf{T} \cdot \mathbf{p} &= \mathbf{T}^{-1} \cdot \mathbf{p}' \\ \mathbf{1} \cdot \mathbf{p} &= \mathbf{T}^{-1} \cdot \mathbf{p}' \\ \mathbf{p} &= \mathbf{T}^{-1} \cdot \mathbf{p}' \end{aligned}$$

wobei $\mathbf{1}$ die 4×4 Einheitsmatrix ist.

Für die in Abschnitt 2.2 vorgestellten Matrizen sehen die inversen Transformationsmatrizen wie folgt aus:

Translationsmatrix:	$\mathbf{T}^{-1}(\mathbf{a}) = \mathbf{T}(-\mathbf{a})$	
Rotationsmatrix:	$\mathbf{R}_{\mathbf{x}}^{-1}(\theta) = \mathbf{R}_{\mathbf{x}}(-\theta) = \mathbf{R}_{\mathbf{x}}^{\top}(\theta);$	$\mathbf{R}_{\mathbf{y}}(\theta)$ und $\mathbf{R}_{\mathbf{z}}(\theta)$ analog
Reflexionsmatrix:	$\mathbf{S}_{\mathbf{x}}^{-1} = \mathbf{S}_{\mathbf{x}};$	$\mathbf{S}_{\mathbf{y}}$ und $\mathbf{S}_{\mathbf{z}}$ analog
Skalierungsmatrix:	$\mathbf{S}^{-1}(s_x, s_y, s_z) = \mathbf{S}(s_x^{-1}, s_y^{-1}, s_z^{-1})$	

5 Transformation

Bei der Rotationsmatrix fällt auf, dass ihre Transponierte auch gleichzeitig ihre Inverse ist. Diese Eigenschaft gilt für alle orthogonalen Matrizen, wozu auch die Rotationsmatrix gehört.

Wenn auf demselben Vektor oder Punkt nacheinander mehrere Transformationen durchgeführt werden, spricht man von einer verketteten Transformation:

$$\mathbf{T}_C = \mathbf{T}_n \cdots \mathbf{T}_2 \cdot \mathbf{T}_1$$

$\mathbf{T}_1$ ist dabei die erste anzuwendende Transformation und $\mathbf{T}_n$ die letzte. Durch Multiplikation der einzelnen Transformationsmatrizen entsteht eine Ergebnismatrix $\mathbf{T}_C$. Mit Hilfe von $\mathbf{T}_C$ kann dann durch eine einzige Matrix-Vektor-Multiplikation die komplette Sequenz an Transformationen auf einen Punkt oder Vektor angewandt werden. Ein Problem ist, dass $\mathbf{T}_C$ zu einer Matrix mit allgemeiner Form degenerieren kann, was das Finden der Inversen $\mathbf{T}_C^{-1}$ erschwert. Bekannte Methoden wie der Gauß-Jordan-Algorithmus [AD14] können dazu verwendet werden, eine Inverse zu berechnen. Einfacher geht es, wenn die Inverse von jeder, in der Verkettung vorkommenden Matrix bekannt ist. Dann lässt sich die von $\mathbf{T}_C$ durchgeführte Transformation durch $\mathbf{T}_C^{-1} = \mathbf{T}_1^{-1} \cdot \mathbf{T}_2^{-1} \cdots \mathbf{T}_n^{-1}$ umkehren:

$$\begin{aligned}\mathbf{T}_C^{-1} \cdot \mathbf{T}_C &= \mathbf{T}_1^{-1} \cdot \mathbf{T}_2^{-1} \cdots \mathbf{T}_n^{-1} \cdot \mathbf{T}_n \cdots \mathbf{T}_2 \cdot \mathbf{T}_1 \\ \mathbf{T}_C^{-1} \cdot \mathbf{T}_C &= \mathbb{1}\end{aligned}$$

In der Implementation lässt sich $\mathbf{T}_C$ und $\mathbf{T}_C^{-1}$ durch folgenden Algorithmus berechnen:

Algorithmus 3 Berechne $\mathbf{T}_C$ und $\mathbf{T}_C^{-1}$

```
1:  $\mathbf{T}_C \leftarrow \mathbb{1}$ 
2:  $\mathbf{T}_C^{-1} \leftarrow \mathbb{1}$ 
3: for each  $\mathbf{T} \in \{\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_n\}$  do
4:    $\mathbf{T}_C \leftarrow \mathbf{T} \cdot \mathbf{T}_C$ 
5:    $\mathbf{T}_C^{-1} \leftarrow \mathbf{T}_C^{-1} \cdot \mathbf{T}^{-1}$ 
```

5.2 Anwendung beim Raytracing

Im vorherigen Abschnitt wurde gezeigt, wie Transformationen funktionieren und wie sie sich umkehren lassen. Nun offenbart sich der Nutzen daraus, die endgültige Transformationsmatrix sowie ihre Inverse vorab zu berechnen. Der Trick besteht beim „Raytracing“ nämlich darin, nicht die eigentlichen Objekte zu transformieren. Schnittpunkt und Normale werden durch Manipulation der Strahlgleichung berechnet. Dazu wird eine geeignete Wahl an Transformationen und Rücktransformationen benötigt. In [Suf07] wird die Vorgehensweise beschrieben.

Es wird zuerst der gewünschte Endzustand betrachtet. Dazu existiere ein Schnittpunkt $\mathbf{r}' = \mathbf{o} + t \cdot \mathbf{d}$ eines vom Kameramodell erzeugten Strahls mit dem transformierten Objekt. Wie erwähnt, lassen sich Objekte nicht so einfach direkt transformieren, Punkte und Vektoren jedoch schon. Da ein Strahl aus einem Startpunkt und einem Richtungsvektor besteht und

5 Transformation

beide leicht zu transformieren sind, ist es das Ziel, das Objekt untransformiert zu lassen und den Strahl zu verändern.

Sei $\mathbf{r}$ der Schnittpunkt aus der modifizierten Strahlgleichung mit dem untransformierten Objekt. $\mathbf{r}'$ erhält man dann durch Transformation von $\mathbf{r}$ mit $\mathbf{T}$:

$$\mathbf{r}' = \mathbf{T} \cdot \mathbf{r}$$

Diese Gleichung lässt sich jetzt so umstellen, dass daraus ersichtlich wird, wie die modifizierte Strahlgleichung auszusehen hat. Die linksseitige Multiplikation mit der inversen Transformationsmatrix $\mathbf{T}^{-1}$ liefert:

$$\begin{aligned}\mathbf{r} &= \mathbf{T}^{-1} \cdot \mathbf{r}' \\ \mathbf{r} &= \mathbf{T}^{-1}\mathbf{o} + t \cdot \mathbf{T}^{-1}\mathbf{d}\end{aligned}$$

wobei $\mathbf{o}$ und $\mathbf{d}$ durch das Kameramodell gegeben sind. Für $\mathbf{r}$ kann nun, wie in Kapitel 4 gezeigt, Parameter t berechnet werden. Dieser wird anschließend in die Strahlgleichung von $\mathbf{r}'$ eingesetzt, um den Schnittpunkt zwischen unverändertem Strahl und transformiertem Objekt zu ermitteln.

5.3 Transformation der Normalen

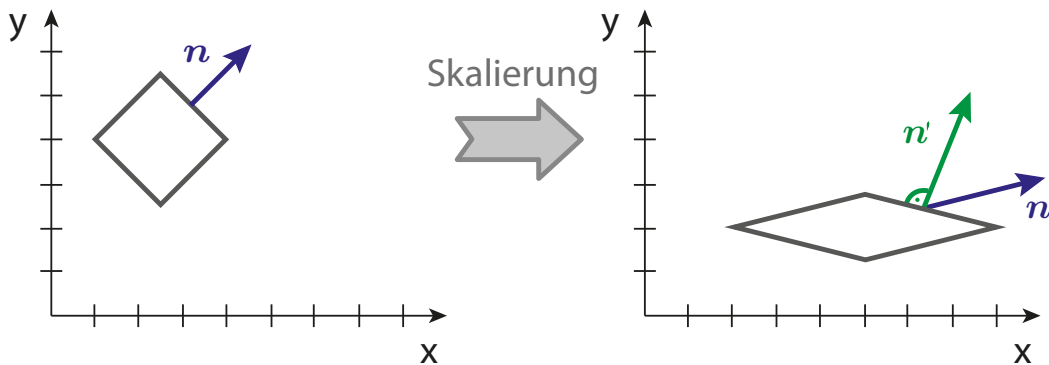


Abb. 5.1: Die Raute und ihre Normale $\mathbf{n}$ werden auf die gleiche Art und Weise transformiert und zwar auf die doppelte Breite und halbe Höhe skaliert. Nach der Skalierung steht $\mathbf{n}$ nicht mehr orthogonal auf der Oberfläche. Die korrekte Normale ist $\mathbf{n}'$. D.h. die Normale muss bei der Transformation gesondert behandelt werden.

Momentan ist der „Raytracer“ in der Lage den Schnittpunkt eines Primärstrahls mit einem transformierten Objekt zu berechnen. Was fehlt, ist die Normale an diesem Schnittpunkt. Problematisch ist die Tatsache, dass die Normalen jeweils nur für die untransformierten Objekte erzeugt werden (s. Kapitel 4). Abb. 5.1 verdeutlicht, dass auf diese Normalen nicht einfach die gleiche Transformation angewendet werden kann, die für das Objekt verwendet

5 Transformation

wurde. Denn nach der Skalierung steht $\mathbf{n}$ in der Abbildung nicht länger senkrecht auf der Oberfläche des Objekts. Die korrekte Normale ist $\mathbf{n}'$.

Nach [Mat10] (S. 87) ist es das Ziel, $\mathbf{n}$ derart zu transformieren, dass dabei $\mathbf{n}'$ herauskommt. Zur Veranschaulichung stellt man sich eine ebene Oberfläche vor, in der zwei Punkte liegen. Die Differenz der Punkte bildet einen Tangentialvektor. Nach einer Transformation liegen die Punkte weiterhin in der ebenen Oberfläche und somit auch der Tangentialvektor. Das heißt, wird auf den Tangentialvektor einer Oberfläche die gleiche Transformation angewandt, wie auf die Oberfläche selbst, bleibt er weiterhin Tangentialvektor.

Nun bedient man sich der Tatsache, dass die Normale $\mathbf{n}$ senkrecht auf dem Tangentialvektor $\mathbf{t}$ steht und damit ihr Skalarprodukt Null ergibt. Nach der Transformation muss auch die neue Normale $\mathbf{n}'$ senkrecht auf dem transformierten Tangentialvektor $\mathbf{t}'$ stehen. Dabei ist $\mathbf{T}$ die Transformation, die auf die Oberfläche und den Tangentialvektor angewendet wird, und $\mathbf{S}$ ist die gesuchte Transformation, die auf die Normale anzuwenden ist. Somit ist:

$$\begin{aligned} 0 &= \mathbf{t}' \cdot \mathbf{n}' \\ 0 &= (\mathbf{S} \cdot \mathbf{n}) \cdot (\mathbf{T} \cdot \mathbf{t}) \\ 0 &= (\mathbf{S} \cdot \mathbf{n})^\top (\mathbf{T} \cdot \mathbf{t}) && \text{siehe Gleichung 2.1} \\ 0 &= \mathbf{n}^\top (\mathbf{S}^\top \cdot \mathbf{T} \cdot \mathbf{t}) \end{aligned}$$

Wie bereits erwähnt, gilt die Prämisse $\mathbf{n}^\top \mathbf{t} = 0$, d.h. $\mathbf{S}^\top \cdot \mathbf{T}$ muss letztendlich einfach aus der Gleichung verschwinden und somit die Einheitsmatrix ergeben. Damit $\mathbf{S}^\top \cdot \mathbf{T} = \mathbf{1}$ gilt, muss $\mathbf{S}^\top = \mathbf{T}^{-1}$ sein. Eine letzte Umformung liefert das Ergebnis $\mathbf{S} = (\mathbf{T}^{-1})^\top$. Um eine Normale also korrekt zu transformieren, muss sie mit der transponierten Inversen der ursprünglichen Transformationsmatrix transformiert werden.

6 Schattierung

Bis jetzt ist der „Raytracer“ in der Lage Strahlen in eine Szene bestehend aus Objekten auszusenden und Schnittpunkte zu berechnen. An diesen Schnittpunkten muss nun ein Farbwert berechnet werden, der anschließend dem PEE zugewiesen wird, welches vom Strahl durchdrungen wurde. Den Vorgang zur Berechnung des Farbwerts bezeichnet man in der Computergrafik als Schattierung.

6.1 Die Rendergleichung

Beim „Raytracing“ erfolgt die Schattierung eines Objekts durch Lösung der Rendergleichung. Der Farbwert ergibt sich aus der Summe aller, über der Hemisphäre eines Schnittpunktes, einfallender Strahldichten. Dabei entscheidet der Einfallswinkel und der Schnittpunkt, welche Frequenzanteile, einer einfallenden Strahldichte, mit welcher Intensität entlang eines Primärstrahls auf das PEE reflektiert wird. Zur Modellierung dieses Sachverhalts verwendet man bidirektionale Reflektanzverteilungsfunktionen (engl. bidirectional reflectance distribution function, BRDF). Die BRDF ist damit maßgeblich für die Oberflächeneigenschaft und somit für die Schattierung eines Objekts verantwortlich.

$$L_o(\mathbf{p}, \boldsymbol{\omega}_o) = L_e(\mathbf{p}, \boldsymbol{\omega}_o) + \int_{2\pi^+} f_r(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \cdot L_i(\mathbf{p}, \boldsymbol{\omega}_i) \cos(\theta_i) d\boldsymbol{\omega}_i \quad (6.1)$$

Gleichung 6.1 zeigt die Rendergleichung. Der Term $L_o(\mathbf{p}, \boldsymbol{\omega}_o)$ steht dabei für die, aus der Oberfläche austretende Strahldichte. Dieser Term ist wie bereits erwähnt abhängig vom Schnittpunkt $\mathbf{p}$ und der Strahlrichtung $\boldsymbol{\omega}_o$, die der inversen Richtung des Primärstrahls entspricht. Der Term $L_e(\mathbf{p}, \boldsymbol{\omega}_o)$ wird für Oberflächen benötigt, die selbst Licht emittieren. Der restliche Summenteil der ausgehenden Strahldichte ergibt sich, indem man über alle möglichen Strahlrichtungen $\boldsymbol{\omega}_i$ in der Hemisphäre integriert, die sich über dem Schnittpunkt $\mathbf{p}$ befindet. Die einfallenden Strahldichten $L_i(\mathbf{p}, \boldsymbol{\omega}_i)$, werden wie erwähnt durch die BRDF $f_r(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o)$ skaliert. Zuletzt fehlt noch der Ausdruck $\cos(\theta_i)$, der dafür sorgt, dass flacher einfallende Strahldichten einen geringeren Beitrag zur ausgehenden Strahldichte leisten, als steiler einfallende. Der Winkel θ_i lässt sich aus der Normale $\mathbf{n}$ am Schnittpunkt $\mathbf{p}$ und der Strahlrichtung $\boldsymbol{\omega}_i$, eines einfallenden Strahls, berechnen.

Die Rendergleichung kann in der Regel nicht analytisch gelöst werden und muss daher approximiert werden. Eine Variante besteht darin, Strahlen gemäß einer Verteilung (z.B. gleichverteilt) über der Hemisphäre, in die Szene zu schicken, um konkrete Werte für $L_i(\mathbf{p}, \boldsymbol{\omega}_i)$ zu berechnen. Trifft einer dieser Strahlen auf ein Objekt wird das Verfahren rekursiv fortgesetzt, bis eine Lichtquelle getroffen wird. Dieses Vorgehen nennt man „Path Tracing“ und kann für kleine Lichtquellen sehr lange dauern, da Strahlen diese nur selten treffen. Für unendlich viele ausgesandte Strahlen, stellt „Path Tracing“ jedoch eine korrekte Lösung der Rendergleichung da.

Beim „Whitted Raytracing“ wird hingegen für jeden Primärstrahl, der ein Objekt trifft, der Term $L_i(\mathbf{p}, \boldsymbol{\omega}_i)$ berechnet, indem Strahlen nur zu allen in der Szene befindlichen Lichtquellen ausgesandt werden. Diese Methode ist deutlich schneller, stellt jedoch nur eine sehr ungenaue Approximation der Rendergleichung da. Die folgenden Abschnitte zeigen, wie Objekte nach der „Whitted“ Methode schattiert werden können.

6.2 Matte Oberfläche

Matte oder „lambertsche“ Oberflächen besitzen eine sehr einfache Schattierung. Sie bestehen im Wesentlichen aus zwei Komponenten und zwar aus einem ambienten und einem diffusen Term:

$$f_{\text{matt}}(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = k_a \cdot c_a + k_d \cdot c_d$$

Der ambiente Term ist unabhängig vom Austrittswinkel ω_o und dem Einfallswinkel ω_i . Da beim „Whitted Raytracing“ die Lichtausbreitung zwischen nicht-Licht emittierenden Objekten fehlt, soll er den Eindruck von vorhandenem Umgebungslicht vermitteln. Bei k_a handelt es sich um den ambienten Reflexionskoeffizienten k_a . Er bestimmt den Anteil der ambienten Komponente in der ausgehenden Strahldichte. Die

Materialkonstante c_a , sorgt für die Färbung des Objekts, indem sie bestimmte Spektren absorbiert und andere reflektiert. Wird c_a als RGB-Wert dargestellt, so entspricht dies einer Multiplikation mit dem RGB-Wert der einfallenden Strahldichte. Für die diffuse Komponente sind die Bedeutungen von k_d und c_d analog. In der gerasterten Computergrafik wird für die matte Oberflächen noch ein Term benötigt, der die Intensität eines Farbwertes skaliert, je nachdem wie flach oder steil ein Strahl von der Lichtquelle auf die Oberfläche trifft. Beim „Raytracing“ ist dieser Sachverhalt bereits durch $\cos \theta_i$, in der Rendergleichung, modelliert

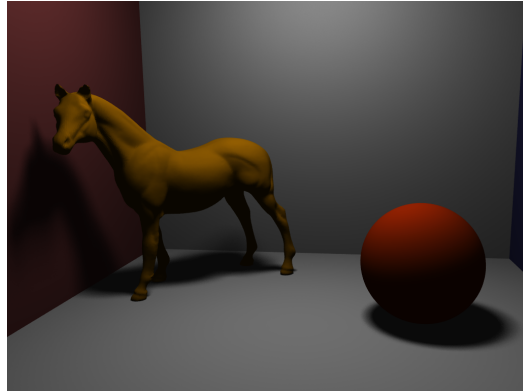


Abb. 6.1: Pferdmodell und Kugel in einer Box

6.3 Spekulare Oberfläche - Phong

Für spekulare Oberflächen muss die matte Schattierung um eine weitere Komponente ergänzt werden. Diese Komponente ist verantwortlich für die Darstellung von Glanzpunkten und wird Phong-Komponente genannt:

$$f_{\text{phong}}(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = f_{\text{matt}}(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) + \left(\frac{\boldsymbol{\omega}_o \cdot \mathbf{r}}{|\boldsymbol{\omega}_o| \cdot |\mathbf{r}|} \right)^n \cdot k_p \cdot c_p$$

Bei den Konstanten k_p und c_p handelt es sich um den Reflexionskoeffizienten und die Materialkonstante. Diese werden mit einem Term multipliziert, der den Winkel zwischen ausgehender Strahlrichtung $\boldsymbol{\omega}_o$ und einem Vektor $\mathbf{r}$ berechnet. Bei $\mathbf{r}$ handelt es sich um die perfekte Reflexionsrichtung, eines einfallenden Lichtstrahls. Er wird erzeugt durch

$\mathbf{r} = -\boldsymbol{\omega}_i \cdot 2 \cdot (\mathbf{n} \cdot \boldsymbol{\omega}_i) \cdot \mathbf{n}$, wobei Normale $\mathbf{n}$ und $\boldsymbol{\omega}_i$ normalisiert sein müssen. Je weiter der Winkel zwischen Beobachter und perfekt reflektiertem Lichtstrahl ist, desto geringer ist der Anteil der Phong-Komponenten an der ausgehenden Strahldichte. Der Exponent n wird dazu verwendet die Ausdehnung der Glanzpunkte festzulegen, wie Abb. 6.2 und Abb. 6.3 zeigen. Je größer n ist, desto konzentrierter erscheinen die Glanzpunkte.

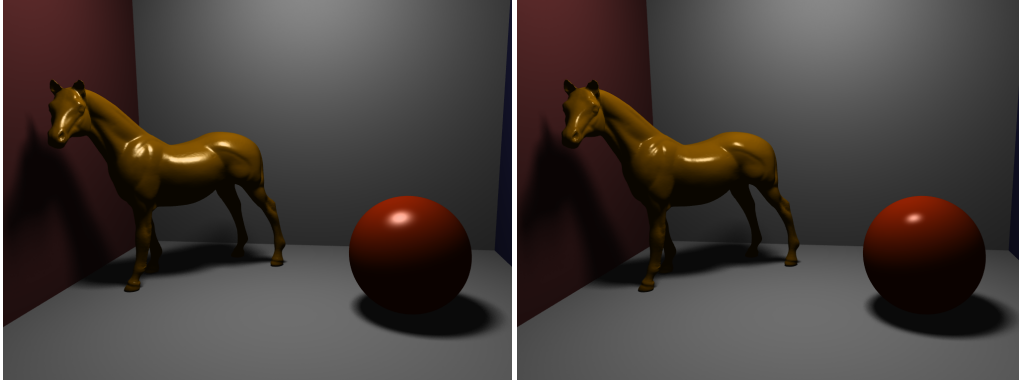


Abb. 6.2: Pferd und Kugel mit spekulativer Oberfläche. Die Größen der Glanzpunkte ergeben sich durch den Exponenten $n = 30$.

Abb. 6.3: Pferd und Kugel mit spekulativer Oberfläche. Die Größen der Glanzpunkte ergeben sich durch den Exponenten $n = 100$.

6.4 Spiegelnde Oberfläche

Die Spiegeloberflächen aus Abb. 6.4 und Abb. 6.5 stellen eine Erweiterung der Phong-Schattierung da. Jedoch sollten die ambiente, diffuse und Phong-Komponente sehr dezent eingesetzt werden. Perfekte Reflexion kann umgesetzt werden, indem ein Strahl in Richtung perfekter Reflexionsrichtung $\mathbf{r}$ ausgesandt wird. Trifft er ein anderes Objekt, wird dort am Schnittpunkt die Schattierungsberechnung durchgeführt und die Strahldichte berechnet. Die dort berechnete Strahldichte kann dann anhand eines weiteren additiven Terms bestehend aus Reflexionskoeffizient und Materialkonstante verändert und zur ausgehenden Strahldichte der perfekt spiegelnden Oberfläche hinzugefügt werden.

Abb. 6.2 zeigt eine matt spiegelnde Oberfläche. Dies erreicht man, indem man Strahlen konisch verteilt, entlang von $\mathbf{r}$ in die Szene schickt.

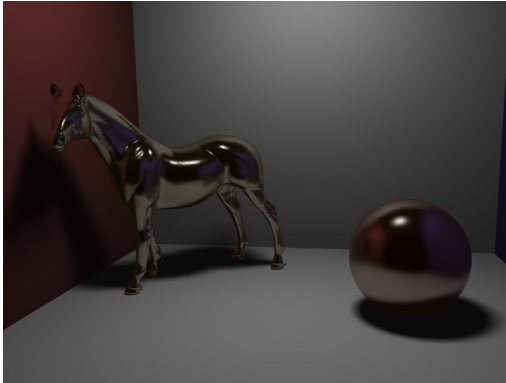


Abb. 6.4: Das Aussehen der matt spiegelnden Oberflächen wird erzeugt, indem Strahlen konisch um die perfekte Reflexionsrichtung $\mathbf{r}$ in die Szene geschickt werden.

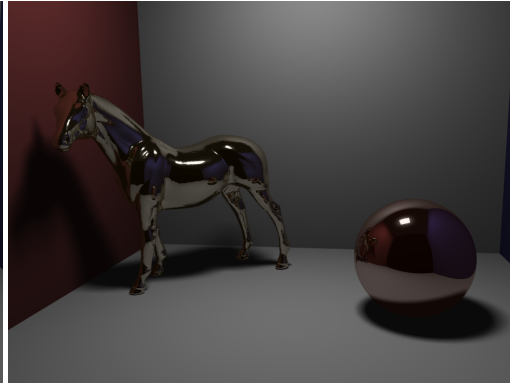


Abb. 6.5: Das Aussehen der perfekt spiegelnden Oberflächen wird erzeugt, indem Strahlen entlang der perfekten Reflexionsrichtung $\mathbf{r}$ in die Szene geschickt werden.

6.5 Dielektrikum

Bei Dielektrika handelt es sich um Materialien, die Anteile eines eintreffenden Lichtstrahls brechen und reflektieren. Jedes Dielektrikum besitzt einen Brechungsindex. Mit Hilfe des Snelliusschen Brechungsgesetz und des Brechungsindex kann bestimmt werden, in welche Richtung ein Strahl gebrochen wird, wenn er vom umgebenden Medium in das Dielektrikum übergeht. Wenn ein Strahl zu flach auf die Oberfläche auftrifft, kann es passieren, dass er komplett reflektiert wird und muss dann wie bei perfekt spiegelnden Oberflächen behandelt werden. Man spricht in diesem Fall von Totalreflexion. Findet keine Totalreflexion statt, wird ein Teil des Strahls gebrochen und ein Teil reflektiert. Durch die Fresnelschen Formeln kann dann bestimmt werden, aus welchen Anteilen der Strahldichte des gebrochenen Strahls und des reflektierten Strahls, sich die ausgehende Strahldichte zusammensetzt.

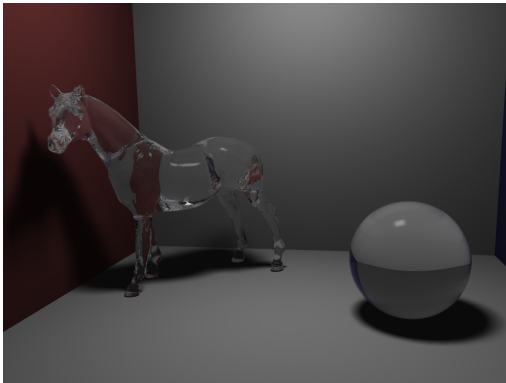


Abb. 6.6: Dielektrika mit einem Brechungsindex von 1,3. Dies entspricht näherungsweise dem Übergang von Luft zu Eis

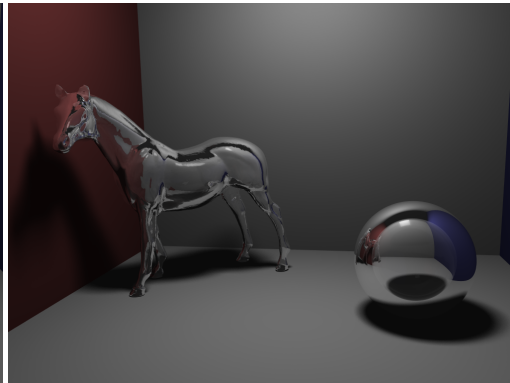


Abb. 6.7: Dielektrika mit einem Brechungsindex von 0,75. Dies entspricht näherungsweise dem Übergang von Wasser zu Luft

7 Triangulierte Objekte

Komplexe Objekte wie der Stanford Bunny bestehen aus einem Polygonnetz. Die einfachste Form des Polygons ist das Dreieck und die so zusammengesetzten Objekte sind triangulierte Objekte. Im Abschn. 4.4 wurde bereits gezeigt wie man den Schnittest und die Berechnung der Normalen für Dreiecke durchführt. Die naive Erweiterung um triangulierte Objekte beim „Raytracing“ verwenden zu können, besteht darin, für jedes einzelne Dreieck im Dreiecksnetz eben diese Schritte durchzuführen. Wenn das triangulierte Objekt nur aus wenigen Dreiecken besteht, ist diese Vorgehensweise noch durchführbar. Der Stanford Buddha besteht jedoch aus über einer Million Dreiecke, wovon jedes mit jedem Strahl getestet werden müsste. In Kapitel 8 werden deshalb Beschleunigungsstrukturen vorgestellt, um die Anzahl durchgeführter Schnittests zu reduzieren.

7.1 Das OBJ-Format

Zur Repräsentation eines Dreiecksnetzes existieren Dateiformate wie das OBJ-Format von Wavefront [Red15]. Das OBJ-Format erlaubt eine kompakte Darstellung des Polygonnetzes, indem alle vorkommenden Punkte aufgelistet werden und die jeweiligen Polygone durch Referenzierung der Punktindices definiert werden. Dadurch werden Punkte, die von mehreren Polygonen verwendet werden, nicht mehrmals gespeichert.

Das nun folgende Beispiel zeigt die Definition eines in der xy-Achse liegenden Quadrats. Es ist im Ursprung zentriert, hat eine Kantenlänge von 2 und ist aus zwei Dreiecken zusammengesetzt.

```
# Definition eines Quadrats im OBJ Format

v -1.0 1.0 0.0
v -1.0 -1.0 0.0
v 1.0 1.0 0.0
v 1.0 -1.0 0.0
# 4 Punkte

vn 0.0000 -0.0000 1.0000
# 1 Normale

f 1//1 2//1 3//1      # Punktindex/Texturkoordinate/Normalenindex
f 4//1 3//1 2//1      # Punktindex/Texturkoordinate/Normalenindex
# 2 Dreiecksflächen
```

Die OBJ-Datei wird von oben nach unten abgearbeitet. Kommentare werden durch `#` gekennzeichnet. Definitionen werden durch Bezeichner eingeleitet und erhalten einen impliziten Index beginnend mit 1. Als erstes werden Punkte definiert, eingeleitet durch den Bezeichner `v`. Die nachfolgenden Zahlen stehen für die x, y und z-Komponente eines Punktes im dreidimensionalen Raum. Der erste definierte Punkt erhält den Index 1. Für folgende Punkte wird der Index jeweils inkrementiert. Als nächstes folgt die Definition von Punktnormalen, gekennzeichnet durch den Bezeichner `vn`. Die drei nachfolgenden Zahlen entsprechen wieder den einzelnen Komponenten. Die Angabe von Punktnormalen ist optional. Sollten keine Punktnormalen definiert sein, wird für jedes Dreieck eine Dreiecksnormale berechnet, wie in Abschn. 4.4 gezeigt.

Zuletzt folgt die Definition der Polygone, in diesem Fall Dreiecke, durch `f`. Die nachfolgenden Spalten folgen dem Schema `Punktindex/Texturkoordinatenindex/Normalenindex`. Im angegebenen Beispiel wird das erste Dreieck somit aus dem ersten, zweiten und dritten Punkt erzeugt. Texturkoordinaten werden zur Einfachheit im Beispiel ausgelassen. In Kapitel 9 wird aber noch konkret gezeigt, wie Texturkoordinaten aussehen. Zuletzt wird jedem Dreieckspunkt eine optionale Punktnormale durch `Normalenindex` zugewiesen. Falls keine Punktnormalen in der OBJ-Datei enthalten sind, können diese nachträglich berechnet werden wie der nachfolgende Abschnitt zeigt.

Das vorgestellte (Minimal-) Beispiel ist so einfach, dass es noch ohne Probleme händisch erstellt werden kann. Für sehr komplexe Objekte, die z.B. aus mehreren zehntausend Polygonen bestehen und mehrere Texturen verwenden, benötigt man eine 3D-Modellierungssoftware zur Erstellung einer solchen Datei, weshalb „3ds Max“ von „Autodesk“ zur Erstellung und Bearbeitung von 3D-Modellen benutzt wird. Zum Laden von 3D-Modellen kommt neben dem selbstgeschriebenen OBJ-Parser noch die quelloffene Bibliothek „Assimp“ zum Einsatz, die eine Vielzahl an Formaten für 3D-Modelle unterstützt.

7.2 Facettierte und glatte Schattierung

Abb. 7.1 zeigt den „Stanford Bunny“ in facettierter Schattierung. Diese Art der Schattierung erhält man, wenn man pro Dreieck eine einzige Dreiecksnormale verwendet. Die einzelnen Dreiecke sind dann gut erkennbar, da sich die Farbe innerhalb eines Dreiecks nicht großartig verändert. Dies hat folgenden Grund: Unabhängig von der Lage des Schnittpunkts wird immer die gleiche Dreiecksnormale verwendet. Die einzige, für die Schattierung relevante Komponente, die sich mit der Lage des Schnittpunkts ändert, ist der Einfallswinkel der Lichtstrahlen. Bei entsprechend kleinen Dreiecken ändert sich der Einfallswinkel des Lichtstrahls nur geringfügig, wodurch der Eindruck entsteht, dass das Dreieck einfarbig ist. Um den „Bunny“ so zu schattieren, wurde eine OBJ-Datei geladen, die lediglich Polygone sowie entsprechende Punkte enthält. Da im OBJ-Format die Polygone mit einem Umlaufsinn gegen den Uhrzeigersinn definiert werden, kann eine Dreiecksnormale direkt nach der Vorgehensweise aus Abschn. 4.4 berechnet werden.

Im Gegensatz dazu zeigt Abb. 7.2 wie eine glatte Schattierung erzielt werden kann, indem sich die Normale abhängig vom Schnittpunkt ändert. Dies geschieht durch Interpolation der Punktnormalen. Seien $\mathbf{n}_0$, $\mathbf{n}_1$ und $\mathbf{n}_2$ die Punktnormalen, die den Dreieckspunkten (z.B. durch die OBJ-Datei) zugewiesen sind. Bei Durchführung des Dreieckschnittpunkts erhält man die baryzentrischen Koordinaten β und γ , die die Position des Schnittpunkts innerhalb des

7 Triangulierte Objekte

Dreiecks widerspiegeln (siehe Abschn.4.4). Diese Koordinaten werden nun zur Berechnung der Normalen am Schnittpunkt $\mathbf{n}_r$ benutzt:

$$\mathbf{n}_r = (1 - \beta - \gamma) \cdot \mathbf{n}_0 + \beta \cdot \mathbf{n}_1 + \gamma \cdot \mathbf{n}_2$$

Wie bereits erwähnt, ist die Angabe der Punktnormalen optional und so ist es möglich, dass keine in der OBJ-Datei vorhanden sind. Es gibt jedoch die Möglichkeit, diese nachträglich zu berechnen. Falls keine Punktnormalen vorhanden sind, wird für jedes Dreieck eine Dreiecksnormale berechnet. Möchte man nun für einen bestimmten Punkt die Punktnormale berechnen, so iteriert man über alle angrenzenden Dreiecke, die diesen Punkt verwenden, und summiert deren Dreiecksnormalen auf. Die resultierende Normale ist jedoch kein Einheitsvektor mehr und muss normalisiert werden, falls diese Eigenschaft gefordert ist.

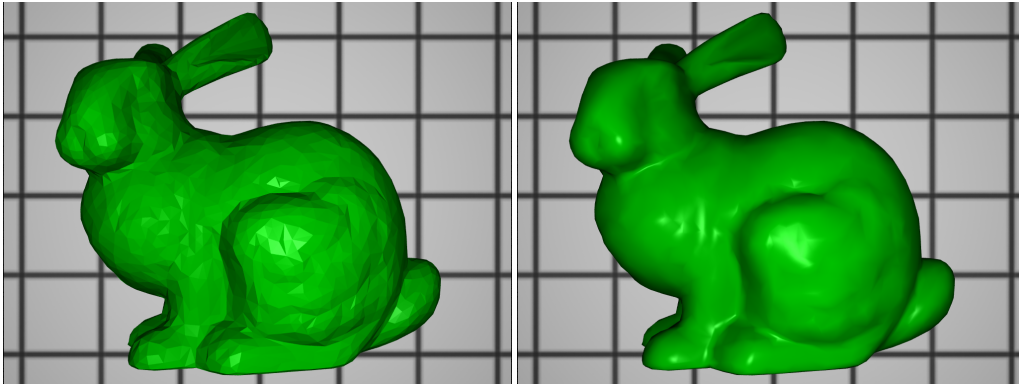


Abb. 7.1: Verwendung nur einer Normalen pro Dreieck führt zu facettierter Schattierung

Abb. 7.2: Einbeziehung und Mittelwertbildung benachbarter Dreiecksnormalen sorgt für glattere Schattierung

7.3 Instanziierung

Durch Instanziierung kann ein einzelnes Polygonnetz für mehrere Objekte verwendet werden. Programmiertechnisch erzeugt man dazu eine Klasse, die eine Referenz auf das Polygonnetz enthält. Des weiteren speichert die Klasse eine Transformationsmatrix und deren Inverse, sowie Oberflächeneigenschaften in Form eines Materials. Mit dieser Ausstattung ist es möglich, zu prüfende Strahlen zu transformieren und den transformierten Strahl mit dem Polygonnetz zu testen. Dadurch lässt sich ein möglicher Schnittpunkt ermitteln. Die dabei erhaltene Normale wird entsprechend transformiert und zur Berechnung der Schattierung, durch das ebenfalls gespeicherte Material, verwendet. So lässt sich auf speicherplatzsparende Art ein und dasselbe Polygonnetz mit unterschiedlichen Transformationen und Schattierungen darstellen. Zudem wird die Laufzeit verkürzt, da das Polygonnetz nicht mehrmals geladen werden muss. Noch schwerer ins Gewicht fällt das Erzeugen einer Beschleunigungsstruktur, was dadurch ebenfalls reduziert wird (s. Unterabschnitt 8.2.3).

8 Schnellere Schnitttests durch Beschleunigungsstrukturen

Bisher wurde jeder Primärstrahl mit jedem schnitttestfähigen Objekt in der Szene getestet. Spätestens mit der Einführung von triangulierten Objekten, die durchaus aus mehreren hunderttausend Dreiecken bestehen können, ist diese Vorgehensweise nicht mehr praktikabel, da so die Laufzeit linear mit der Anzahl an schnitttestfähigen Objekten zunimmt. Mit den folgenden Beschleunigungsstrukturen lässt sich die Anzahl der Schnitttests durch eine obere Grenze von $O(\log(n))$ beschränken, wobei n die Anzahl schnitttestfähiger Objekte ist. Der Fokus wird dabei auf die „bounding interval hierarchy“ (BIH) [WK06] gelegt.

8.1 Übersicht

Es existieren bereits verschiedene Ansätze, um die Schnitttests mit der Geometrie zu beschleunigen. Häufig geht es darum, frühzeitig die Objekte auszuschließen, mit denen sicher kein Schnitt vorliegt. Ein erster Schritt, um dies zu erreichen, gelingt durch die Verwendung von Hüllkörpern. Das sind meist sehr einfache Körper wie Kugeln oder Quader, für die ein Schnitttest sehr schnell durchgeführt werden kann. Schneidet ein Strahl den Hüllkörper nicht, so muss auch nicht die umschlossene Geometrie überprüft werden. Schneidet jedoch ein Strahl den Hüllkörper, muss die komplette Geometrie auf Schnitte überprüft werden. Falsch-positive Fälle liegen vor, wenn ein Hüllkörper geschnitten wird, aber kein Schnitt mit der Geometrie gefunden wird. Um solche unnötigen Tests zu vermeiden, sollte der Hüllkörper die Geometrie möglichst eng umschließen.

Ihr volles Potential entfalten Hüllkörper in Kombination mit räumlichen Hierarchien (engl. spatial hierarchy). Dazu gehört z.B. die „BVH“ (engl. „bounding volume hierarchy“). Ziel ist es, die gesamte Geometrie entsprechend ihrer Lage im Raum zu sortieren und in einer Baumstruktur abzulegen. Man beginnt an der Wurzel und erzeugt einen Hüllkörper für die gesamte Geometrie. Nun teilt man die Geometrie anhand eines bestimmten Kriteriums auf, z.B. gleichmäßige Aufteilung der Geometrie auf beide Kindknoten und erzeugt für die aufgeteilte Geometrie kleinere Hüllkörper, die in den Kindknoten gespeichert werden. Dies wird solange fortgesetzt, bis ein Abbruchkriterium erfüllt ist, wie z.B. maximale Tiefe des Baumes oder eine bestimmte Anzahl an Objekten im Knoten, für die keine weitere Teilung durchgeführt wird. Knoten, die eine dieser Bedingungen erfüllen, nennt man Blätter. In ihnen werden Referenzen auf diese Objekte gespeichert. Die Schwierigkeit besteht darin, die Geometrie so aufzuteilen, dass bei der Traversierung des Baumes möglichst nur ein Pfad bearbeitet werden muss. Die Traversierung des Baums beginnt an der Wurzel, mit dem größten Hüllkörper und wird rekursiv mit Kindknoten fortgesetzt, die ebenfalls vom Strahl geschnitten werden.

Einen anderen Ansatz verfolgen die „kd-Trees“. Hier geht es darum, den Raum so aufzuteilen (engl. space partitioning), dass bei der Traversierung des „kd-Trees“ möglichst große Raumteile ausgeschlossen werden können und somit die darin befindlichen Objekte nicht getestet werden müssen. Kritisch für die Laufzeit ist hierbei die Wahl der Trennebenen, die den Raum rekursiv in zwei kleinere Bereiche aufteilen. Bei der Traversierung wird getestet, welcher der beiden Raumteile bzw., ob beide Raumteile vom Strahl durchdrungen werden, indem ein Strahl-Ebenen Schnittpoint mit der Trennebenen durchgeführt wird. Sobald man einen Raumteil erreicht, der nicht mehr weiter unterteilt ist, werden die darin befindlichen Objekte mit dem Strahl auf Schnitte getestet.

8.2 Die „bounding interval hierarchy“

Die Entscheidung auf die „bounding interval hierarchy“ (BIH) als Beschleunigungsstruktur zu setzen, beruht auf mehreren Gründen.

Für kd-Tree kann vorab nicht entschieden werden, wieviel Speicherplatz im RAM benötigt wird, da Objekte in mehreren Raumteilen referenziert werden können. Denn der Raum kann so geteilt werden, dass ein Objekt in beiden Seiten des geteilten Raumes liegt. Des weiteren birgt das die Problematik von numerischen Instabilitäten und erfordert evt. die Verwendung eines Benachrichtigungssystems, um Objekte nicht mehrfach auf Schnitte überprüfen zu müssen.

BVH sind, was die Performance anbelangt, den gut konstruierten kd-Trees unterlegen, aber dafür recht einfach zu bauen. Der Speicherbedarf kann besser abgeschätzt werden (linear in anzahl der Objekten). Sie sind numerisch stabiler, da kein Strahl-Ebenen Schnitt, sondern nur min-max Operationen angewendet werden.

Die BIH vereinigt viele der Vorteile beider Methoden, ohne unter deren Nachteile zu leiden. So sind BIH sehr leicht zu verstehen und zu implementieren, was die Fehleranfälligkeit reduziert. Ihre Konstruktion ähnelt der von BVH, wodurch sie sehr schnell erzeugt werden kann. Damit ist sie für dynamische Szenen oder Echtzeit „Raytracing“ verwendbar. Die Traversierung verläuft hingegen sehr ähnlich zu der von kd-Trees. Damit ist die BIH nicht nur konkurrenzfähig zu den Traversierungszeiten von kd-Trees, sondern schlägt diese sogar in bestimmten Szenen [WK06]. Jedes Objekt wird exakt einmal in einem Blatt referenziert. Dadurch ist der benötigte Speicherverbrauch vorab berechenbar und es werden keine Benachrichtigungssysteme benötigt.

8.2.1 Konstruktion

Zur Konstruktion der BIH werden „axis aligned bounding boxes“ (AABB) benötigt. Das sind quaderförmige Hüllkörper, deren Kanten parallel zu den Achsen des globalen Koordinatensystems verlaufen. Die Repräsentation einer AABB gestaltet sich auf Grund dieser Eigenschaft relativ einfach. Es reicht aus für jede Achse die Ausdehnung der AABB durch einen minimalen und maximalen Wert festzulegen. Die Bezeichner dafür lauten $min_{x,y,z}$, $max_{x,y,z}$, wobei das Subskript die entsprechende Achse referenziert.

Ein schneller Strahl-AABB Schnittpoint (s. Alg. 4) wird in [Mat10] vorgestellt. Dieser liefert, für den Parameter t aus der Strahlgleichung, die Werte t_{min} für den nächsten Schnittpoint und t_{max} für den entfernten. Es sei erwähnt, dass der Code so konstruiert wurde,

dass er die Besonderheiten der IEEE Spezifikation für Gleitkommazahlen berücksichtigt. So ist die Ausführung des Codes auch dann noch korrekt, wenn eine Komponente $d.i$ des Strahlrichtungsvektor 0 ist und d_{inv} somit die Werte $-\infty$ oder ∞ annimmt. Das Gleiche gilt für den Fall, dass der Strahlursprung in einer der Seiten der AABB liegt. Dann kann t_{near} oder t_{far} in Zeile 6 und 7 den Wert „NaN“ annehmen, da $\mathbf{o}.i \cdot d_{inv}$ zu „ $0 \cdot \pm\infty$ “ wird. Da Vergleichsoperationen mit „NaN“ immer „false“ liefern (außer man vergleicht 2 „NaN“ miteinander) bleiben t_0 und t_1 unverändert.

Algorithmus 4 Schnittest Strahl-AABB

```

1:  $t_0 \leftarrow -\infty$ 
2:  $t_1 \leftarrow \infty$ 
3:
4: for each  $i \in \{x, y, z\}$  do
5:    $d_{inv} \leftarrow 1/d.i$  ▷  $d.i \equiv$  Komponenten des Strahlrichtungsvektors
6:    $t_{near} \leftarrow \min_i -\mathbf{o}.i \cdot d_{inv}$  ▷  $\mathbf{o}.i \equiv$  Komponenten des Strahlursprungs
7:    $t_{far} \leftarrow \max_i \mathbf{o}.i \cdot d_{inv}$ 
8:   if  $t_{near} > t_{far}$  then
9:      $\text{swap}(t_{near}, t_{far})$  ▷ Vertauschen der Werte von  $t_{near}$  und  $t_{far}$ 
10:  if  $t_{near} > t_0$  then
11:     $t_0 \leftarrow t_{near}$ 
12:  if  $t_{far} < t_1$  then
13:     $t_1 \leftarrow t_{far}$ 
14:  if  $t_0 > t_1$  then
15:    return false
16:
17:  $t_{min} \leftarrow t_0$ 
18:  $t_{max} \leftarrow t_1$ 
19: return true

```

Ziel ist es nun, die Objekte der Szene in einer Baumstruktur abzulegen. Es beginnt damit, eine globale AABB zu erzeugen, die alle Objekte der Szene umfasst. Um die Objekte partitionieren zu können, muss eine Trennebene gefunden werden. Das Konstruktionsverfahren der BIH teilt dazu die globale AABB mittig durch sogenannte Trennebenenkandidaten entlang der längsten Achse auf, wie Abb. 8.1 -a zeigt. Dabei werden nur die Seiten weiter unterteilt, in denen sich auch tatsächlich Objekte befinden. So zeigt Abb. 8.1 -b die final verwendeten Trennebenen, leere Räume sind nicht weiter unterteilt.

Nach Festlegung einer Trennebene wird überprüft, welche Objekte sich links bzw. rechts von der Trennebene befinden. Eine einfache Vorgehensweise besteht darin eine AABB um die zu prüfenden Objekte zu legen und ihren Mittelpunkt zu berechnen. Da die Trennebene immer parallel zu einer der 3 Achsen des globalen Koordinatensystems verläuft, kann die entsprechende Komponente des Mittelpunkts verwendet werden, um die Zuordnung durchzuführen. Nun wird ein innerer Knoten erzeugt und dieses Schema rekursiv fortgesetzt, bis eine Abbruchbedingung, wie maximale Tiefe des Baumes oder minimale Anzahl zu teilender Objekte erreicht ist. Ist eine Abbruchbedingung erreicht, wird ein Blatt erzeugt und die vom Raumteil des Blattes umschlossenen Objekte im Blatt vermerkt.

¹Quelle: [WK06] / Seite 5

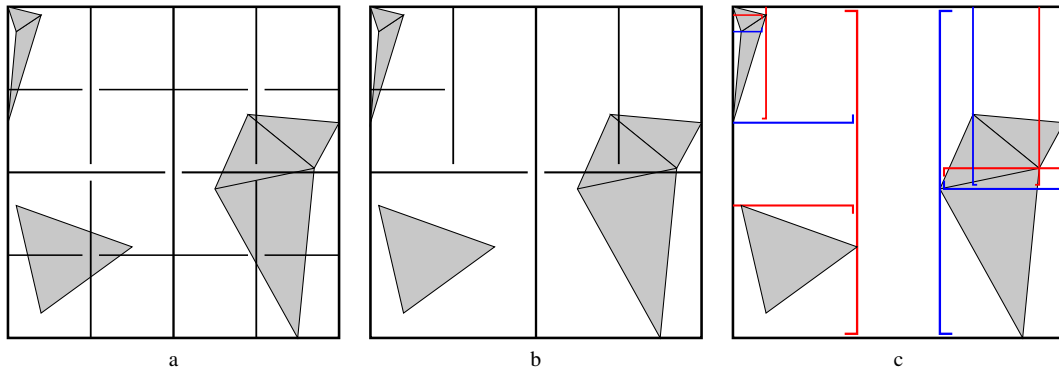


Abb. 8.1: a) Rekursive Unterteilung der Szene anhand von Trennebenenkandidaten. Die jeweils längste Achse wird mittig durch einen Trennebenenkandidat geteilt. b) Tatsächliche Aufteilung des Raums nach Ablegen der Objekte in Blättern. Leere Räume benötigen keine weitere Unterteilung. c) Rote Klammern zeigen die maximale Ausdehnung der Objekte des linken Kindknoten entlang der geteilten Achse. Blaue Klammern zeigen minimale Ausdehnung der Objekte des rechten Kindknoten.¹

Die allgemeine Definition eines Knotens sieht in der Programmiersprache C wie folgt aus und dient zur Erzeugung von inneren Knoten, wie auch Blättern [WK06]. Im Folgenden wird erläutert, welche Informationen in inneren Knoten und Blättern gespeichert werden:

```
typedef struct
{
    int Index; //LSB: (x=00,y=01,z=10)-Achse oder Blatt(11)
    union
    {
        int Items; // Nur Blätter
        float Clip[2]; // Nur innere Knoten
    };
} BIH_Node;
```

Die beiden niederwertigsten Bit (engl. „least significant Bit“ LSB) der Variable „Index“ werden dazu benutzt, anzuzeigen, ob es sich um ein Blatt handelt, falls nicht, handelt es sich um einen inneren Knoten und die beiden LSB zeigen an, entlang welcher Achse des globalen Koordinatensystems die Unterteilung stattfindet. Die Bedeutung der einzelnen Werte kann der Definition entnommen werden. Die restlichen Bit der Variable „Index“ werden dazu verwendet, einen Zeiger auf die beiden Kindknoten zu speichern, falls es sich um einen inneren Knoten handelt. Blätter speichern hingegen einen Zeiger auf eine Liste dem Blatt zugeordneter Objekte. Innere Blätter müssen zudem noch 2 Trennebenen speichern. Dabei handelt es sich nicht um die Trennebenenkandidaten, die zur Partitionierung der Objekte verwendet wurden! Vielmehr wird in Clip[0] die maximale Ausdehnung der Objekte im linken Kindknoten entlang der Koordinatenachse gespeichert und in Clip[1] die minimale Ausdehnung der Objekte im rechten Kindknoten. In Abb. 8.1 -c sind diese Trennebenen durch rote und blaue Klammern gekennzeichnet. Blätter müssen keine Trennebenen speichern, deshalb wird ein „union“ verwendet, um den ohnehin verbrauchten Speicherplatz für das Speichern der Anzahl im Blatt enthaltener Objekte zu verwenden.

8.2.2 Traversierung

Die Traversierung beginnt damit, dass ein Schnitttest gemäß Algorithmus 4 mit der globalen AABB durchgeführt wird. Wenn die globale AABB nicht geschnitten wird, liegt auch kein Schnitt mit der Geometrie vor und die Traversierung endet. Ansonsten erhält man für den Parameter t der Strahlgleichung die Werte t_{min} für den nächsten Schnittpunkt und t_{max} für den entfernten. Jetzt wird der Wurzelknoten der BIH zusammen mit t_{min} und t_{max} auf einen Stapel (engl. „stack“) gelegt und die nachfolgenden Anweisungen durchgeführt, bis der Stapel leer ist oder frühzeitig der nächste Schnitt gefunden wurde, wozu die Variable $t_{Schnitt}$ benötigt wird. Diese wird zu Beginn auf einen sehr großen Wert gesetzt.

1. Entnehme den obersten Knoten und die zugehörigen Werte t_{min} und t_{max} vom Stapel.
2. Teste ob $t_{Schnitt} < t_{min}$, falls ja, fahre mit Anweisung 1. fort. Denn der aktuell zu betrachtende Raumteil ist weiter entfernt als der bisher nächste gefundene Schnittpunkt. Somit können die darin liegenden Objekte unmöglich einen näheren Schnittpunkt liefern.
3. Teste ob es sich beim aktuellen Knoten um ein Blatt handelt, falls ja, führe Schnitttests mit allen enthaltenen Objekten durch und weise $t_{Schnitt}$ den minimalen Strahlgleichungsparameter zu. Fahre mit Anweisung 1. fort.
4. Weise das linke Kind des aktuellen Knotens der Variable $KnotenNah$ und das rechte der Variable $KnotenFern$ zu. Teste ob die Komponente der Strahlrichtung $\mathbf{d}$, die der Trennebene des aktuellen Knotens entspricht, negativ ist. Falls ja, weise $KnotenNah$ das rechte und $KnotenFern$ das linke Kind zu. Somit ist sicher gestellt, dass der Raumteil, der später vom Strahl geschnitten wird, zuerst in den Stapel gelangt und damit nach dem Raumteil betrachtet wird, der zuerst geschnitten wird.
5. Führe einen Strahl-Ebenen Schnitt mit $cNah = (\mathbf{Clip}[0] - \mathbf{p}.i) * \mathbf{d}.i^{-1}$ durch, wobei $\mathbf{Clip}[0]$ die Position der Trennebene des linken Teilraumes beinhaltet (s. Definition BIH.Node). $\mathbf{p}.i$ und $\mathbf{d}.i$ sind die Komponenten des Strahlursprungs und der Strahlrichtung, wobei i die Achse ist, auf die sich die Trennebene des aktuellen Knotens bezieht. Berechne $cFern$ analog, nur mit der Trennebene des rechten Teilraums $\mathbf{Clip}[1]$ anstelle von $\mathbf{Clip}[0]$. Falls $\mathbf{d}.i$ negativ ist, wurde in Anweisung 4. $KnotenNah$ und $KnotenFern$ vertauscht, also tausche auch die Werte von $cNah$ und $cFern$, falls $\mathbf{d}.i$ negativ ist.
6. Teste ob $cFern \leq t_{min}$, falls ja, setze t_{min} auf das Maximum, gebildet aus t_{min} und $cFern$, um den fernen Teilraum weiter einzugrenzen und lege $KnotenFern$ zusammen mit t_{max} und dem angepassten t_{min} auf den Stapel.
7. Teste ob $(0 \leq cNah) \wedge (t_{min} \leq cNah)$, falls ja, setze t_{max} auf das Minimum gebildet aus t_{max} und $cNah$, um den nahen Teilraum weiter einzugrenzen und lege $KnotenNah$ zusammen mit t_{min} und dem angepassten t_{max} auf den Stapel.
8. Überprüfe, ob der Stapel leer ist, falls ja, fahre mit Anweisung 9. fort. Ansonsten fahre mit Anweisung 1. fort.
9. Wenn sich $t_{Schnitt}$ vom initial zugewiesenen Wert unterscheidet, kam ein Schnitt zustande, ansonsten nicht. $t_{Schnitt}$, verwendet als Strahlgleichungsparameter, liefert dann den nächsten Schnitt. Ende.

Ein Detail ist in der beschriebenen Traversierung noch nicht beachtet worden. Und zwar können Strahlen parallel zu den Koordinatenebenen verlaufen, wodurch mindestens eine Komponente der Strahlrichtung $\mathbf{d}$ gleich 0 sein wird. Damit liefert der Schnitttest aus Anweisung 5. die Werte ∞ und $-\infty$ wegen Division durch 0, bzw. „NaN“ wenn die Komponente $\mathbf{p}.i$ des Strahlursprungs mit einer der Trennebenenpositionen aus `Clip[0]` oder `Clip[1]` übereinstimmt, wegen der Berechnung $\frac{0}{0}$. Diese Spezialfälle müssen noch gesondert überprüft werden.

8.2.3 Geschwindigkeitsmessung

Ziel dieses Abschnittes ist es, die Geschwindigkeit der BIH anhand zweier Szenen zu demonstrieren und den Unterschied zur Ausführung des „Raytracers“ ohne Beschleunigungsstruktur zu zeigen. Gemessen wird die Zeit, die die BIH zur Konstruktion benötigt, sowie die Laufzeit des reinen Renderprozesses, dessen größter Anteil in der Traversierung der BIH besteht. Die Messung findet auf einem „Intel i5-3570k“ Prozessor statt. Die zu rendernden Szenen besitzen eine Auflösung von 800×800 Pixel. Jeder Pixel wird anhand eines Primärstrahls durch den Mittelpunkt berechnet.

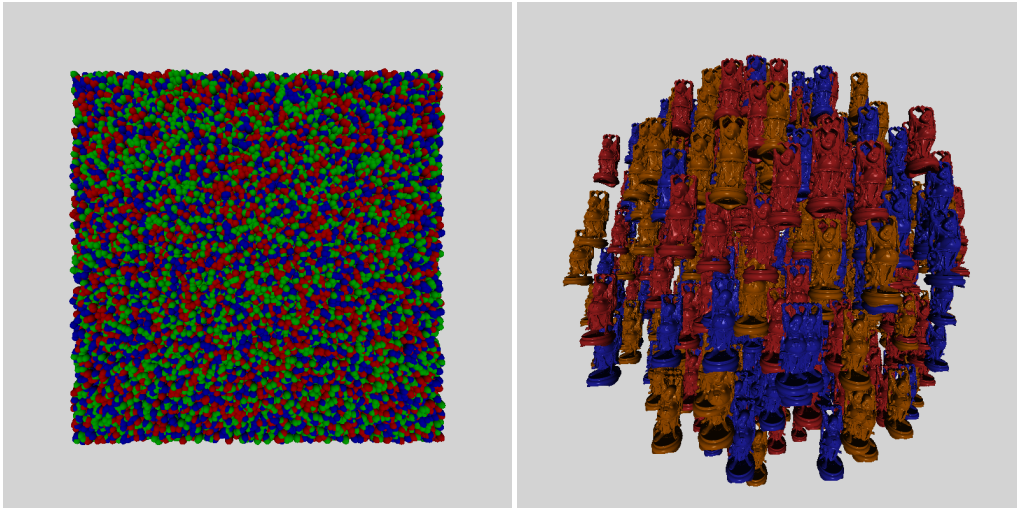


Abb. 8.2: Die Testszene „Kugeln“ zeigt 1 Million, innerhalb eines Würfels mit der Kantenlänge 9, gleichverteilte Kugeln. Die Auflösung des Bilds beträgt 800×800 Pixel.

Abb. 8.3: Die Testszene „Buddhas“ zeigt 1000, innerhalb einer Kugel mit Radius 9, gleichverteilte Stanford Buddhas. Die Auflösung des Bilds beträgt 800×800 Pixel.

In der Testszene „Kugeln“ (s. Abb. 8.2) werden Kugeln innerhalb eines Würfels mit Kantenlänge 9 gleichverteilt. Es wird die Zeit in Sekunden gemessen, die es dauert die BIH zu erzeugen sowie zu traversieren. Die Erzeugung der BIH findet dabei immer „single threaded“ statt, wohingegen die Traversierung mit mehreren „threads“ durchgeführt werden kann. In der Traversierungszeit steckt zudem noch die Zeit, die zur Schattierung der Objekte benötigt wird. Um diese nicht unnötig aufzublähen, wird nur eine Phong-Schattierung verwendet.

Die Messungen aus Abb. 8.4 werden für 1000, 10000, 100000 und 1 Million Objekte durchgeführt. Die Diagrammsegmente unterteilen sich in Konstruktionszeit sowie Traversierungs-

8 Schnellere Schnitttests durch Beschleunigungsstrukturen

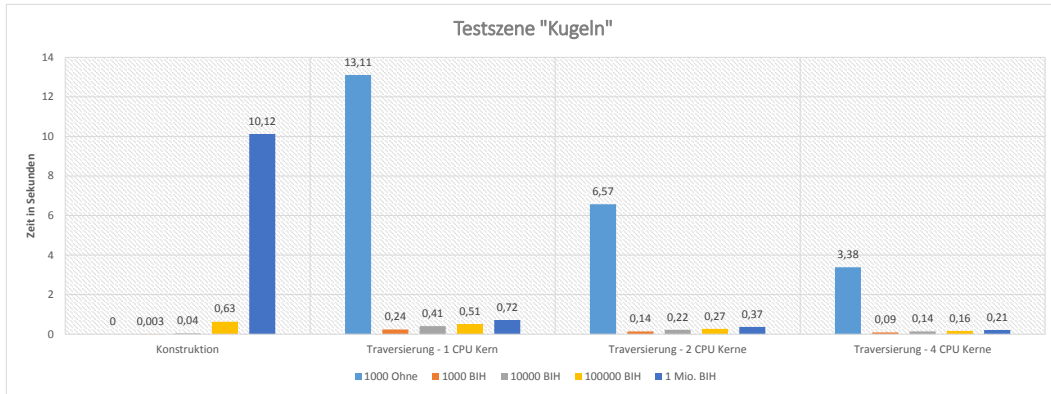


Abb. 8.4: Zeitmessung in Sekunden der Testszene „Kugeln“. Das Diagramm ist unterteilt in Segmente, zur Zeitmessung der Konstruktion sowie der Traversierung der BIH. Die Konstruktion findet immer auf einem CPU Kern statt, wohingegen die Traversierung mit 1, 2 und 4 Kernen durchgeführt wird. Die Balken innerhalb eines Segments stehen für die Anzahl der Objekte in der Szene und zeigen an, ob die BIH verwendet wurde oder nicht („Ohne“).

zeiten, durchgeführt mit unterschiedlicher Anzahl an „threads“. Es zeigt sich bereits bei 1000 Objekten, dass es äußerst unpraktikabel ist jeden Primärstrahl mit jedem Objekt auf Schnitt zu testen. Betrachtet man den Fall, in dem die Konstruktion sowie Traversierung nur auf einem CPU Kern stattfindet, so ist die BIH bei 1 Million Objekte trotz Konstruktionszeit von 10,12 Sekunden und einer Traversierungszeit von 0,72 Sekunden immer noch schneller als der naive Ansatz ohne Beschleunigungsstruktur, der 13,11 Sekunden für 1000 Objekte benötigt.

Für den Stanford Buddha, bestehend aus 100000 Dreiecken, würde das Rendern ohne Beschleunigungsstruktur viel zu lange dauern, weshalb die Zeitmessung nur mit BIH durchgeführt wurde. Die Konstruktionszeiten aus Abb. 8.5 zeigen eine Besonderheit, denn sie bleiben konstant, trotz steigender Anzahl Buddhas in der Szene. Dies ist der bereits angesprochenen Instanziierung geschuldet. Es wird nur einmal das Dreiecksnetz des Buddhas geladen und dafür eine BIH erzeugt. Die verschiedenen Instanzen speichern dann eine Referenz auf das Dreiecksnetz und verwenden unterschiedliche Transformationsmatrizen und Materialien.

Ein weiterer Fakt, der aus den Diagrammen hervorgeht, ist die Tatsache, dass die Konstruktionszeit der BIH in $O(n \cdot \log(n))$ und die Traversierungszeit in $O(\log(n))$ liegt, wobei n die Anzahl schnitttestfähiger Objekte ist. Des weiteren zeigt sich, dass „Raytracer“ prädestiniert dazu sind, parallelisiert zu werden. In der Testszene „Buddhas“ lässt sich die Traversierungszeit für 1000 Buddhas durch die Verwendung von 4 CPU Kernen fast um Faktor 4 verringern (auf 0,87 Sekunden), im Vergleich zur seriellen Ausführung (3,21 Sekunden) (s. Abb. 8.5).

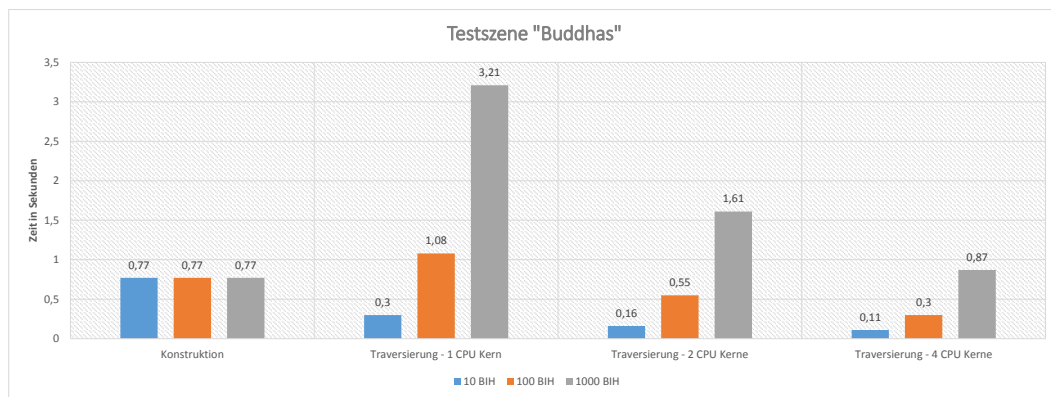


Abb. 8.5: Zeitmessung in Sekunden der Testszene „Buddhas“. Das Diagramm ist unterteilt in Segmente, zur Zeitmessung der Konstruktion sowie der Traversierung der BIH. Die Konstruktion findet immer auf einem CPU Kern statt, wohingegen die Traversierung mit 1, 2 und 4 Kernen durchgeführt wird. Die Balken innerhalb eines Segments stehen für die Anzahl der Objekte in der Szene. Zum Rendern der Buddhas wurde Instanziierung verwendet, wodurch die Konstruktionszeit gleich bleibt.

9 Texturierung

In der Realität besitzen Objekte eine Vielzahl an geometrischen und materiellen Feinstrukturen. Beispiele dafür sind die Maserung bei Holz oder Kratzer und Unebenheiten in Oberflächen. Es wäre viel zu aufwendig, all diese Feinheiten durch ein polygonales Modell nachzubilden. Um Objekte dennoch mit solchen Details ausstatten zu können, verwendet die Computergrafik Texturen.

Abstrakt betrachtet, handelt es sich bei einer Textur um eine Funktion, die einen Punkt in der xy-Ebene auf ein Texturelement, auch Texel genannt, abbildet. Meistens werden Texturen durch Bitmaps realisiert. Diese bestehen aus einem 2D-Array in dem Farbwerte z.B. in RGB-Darstellung gespeichert werden. Daher werden Texel oft mit Farbwerten assoziiert. Tatsächlich können Texel auch andere Dinge repräsentieren wie z.B. Normalen oder Skalare. Dies hängt davon ab, wie Texturen umgesetzt werden und wie ein Texel interpretiert wird. Ein RGB-Farbwert besteht beispielsweise aus 3 Byte, wobei jedes Byte für einen der Farbkanäle Rot, Grün und Blau steht. Diese Bytes können jedoch auch als Komponenten einer Normalen verwendet werden.

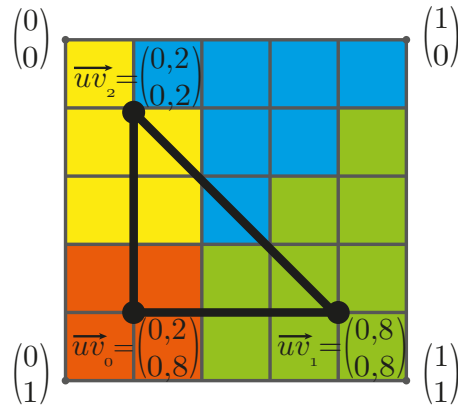


Abb. 9.1: Textur als 2D-Array mit uv-Koordinaten als Spaltenvektor der Form $\begin{pmatrix} u \\ v \end{pmatrix} \in \mathbb{R}^2$

Abb. 9.1 zeigt die schemenhafte Darstellung einer Bildtextur. Im Kontext der Texturierung heißen die Ebenenkoordinaten nicht wie üblich xy-Koordinaten sondern uv-Koordinaten. Diese sind meistens normiert und liegen deshalb im Intervall $[0 \ 1]$. Um nun ein Objekt mit einer Textur zu überziehen, muss eine Abbildung der planaren uv-Koordinaten auf Punkte der Objektoberfläche stattfinden. Diese Abbildung wird auch „Mapping“ genannt. Für einfache geometrische Objekte wie die Kugel kann die Mercator-Projektion verwendet werden, um die Kugeloberfläche auf eine Ebene zu projizieren und somit das Mapping der uv-Koordinaten durchzuführen. Für triangulierte Objekte wurde im Rahmen dieses Projekts die 3D-Modellierungssoftware „3ds Max“ verwendet, um Dreiecksgitter auf eine Ebene zu projizieren und so mit Texturen zu versehen.

Abb. 9.1 zeigt ein Dreieck bei dem das „Mapping“ bereits durchgeführt wurde. Jedem Dreieckspunkt ist eine der uv-Koordinaten uv_0 , uv_1 und uv_2 zugewiesen, wobei diese einen Spaltenvektor der Form $\begin{pmatrix} u \\ v \end{pmatrix} \in \mathbb{R}^2$ repräsentieren. Um nun die Färbung des Dreiecks an einem Schnittpunkt zu bestimmen, führt man den Dreiecksschnittpunkt durch, (s. Abschn. 4.4) um die baryzentrischen Koordinaten β und γ zu erhalten. Durch Interpolation der uv-Koordinaten lässt sich die Koordinate uv_r am Schnittpunkt bestimmen:

$$uv_r = (1 - \beta - \gamma) \cdot uv_0 + \beta \cdot uv_1 + \gamma \cdot uv_2$$

Die Komponenten von uv , können nun dazu benutzt werden, die entsprechenden Array-Indizes zu ermitteln, indem man u mit der Anzahl der Spalten und v mit der Anzahl der Zeilen des Arrays multipliziert (dazu müssen die uv-Koordinaten normalisiert sein).

9.1 Normal Mapping

Neben der Möglichkeit, Objekte mit Farbinformationen zu versehen, können Texturen dazu verwendet werden, um Tiefeninformationen darzustellen, die in der Geometrie nicht vorhanden sind. Dazu können sogenannte „Normal Maps“ verwendet werden. Sie werden typischerweise wie Farbtexturen als Bilder im RGB-Format gespeichert. Jedoch repräsentiert ein Texel keinen konkreten Farbwert, sondern wird als Vektor interpretiert, der dazu verwendet wird, die Ausrichtung der Oberflächennormalen zu verändern, wodurch ein Tiefeneindruck entsteht. Ziel dieser Vorgehensweise ist die Verkürzung von Rechenzeit, indem die Details eines komplexen Modells durch „Normal Maps“ festgehalten und auf ein einfacheres Modell mit weniger Polygonen übertragen werden.

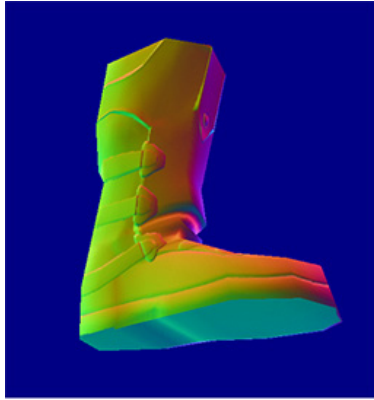
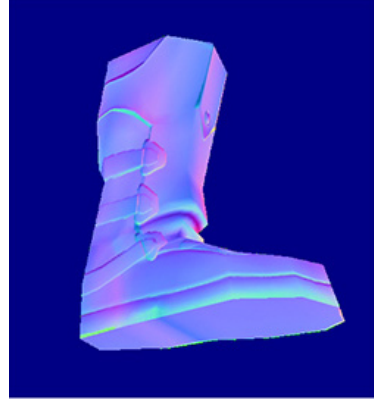
In Abb. 9.2 wurde für die Bildsynthese das Modell eines T-Rex unter Verwendung glatter Schattierung (s. Abschn. 7.2) verwendet. In Abb. 9.3 ist das gleiche Modell unter Verwendung von „Normal Mapping“ zu sehen. Der einzige Unterschied besteht in der Ausrichtung der Oberflächennormalen, wodurch Falten und Vertiefungen vor allem an Kopf und Hals sichtbar werden, wobei die Rechenzeit nur geringfügig von 2,08 auf 2,1 Sekunden steigt.



Abb. 9.2: Das Modell eines T-Rex ohne „Normal Mapping“: Laufzeit 2,08 Sekunden



Abb. 9.3: Das selbe Modell mit „Normal Mapping“: Laufzeit 2,1 Sekunden

Abb. 9.4: Objektraum-“Normal Map“¹Abb. 9.5: Tangentialraum-“Normal Map“¹

Man unterscheidet zwei Arten von „Normal Maps“, die eine bezieht sich auf den Objektraum und die andere auf den Tangentialraum der Geometrie. In Abb. 9.4 sieht man, dass die Objektraum-“Normal Map“ regenbogenfarben ausfällt. Dies liegt daran, dass die Objektraum-“Normal Map“ Vektoren speichert, die direkt als Normalen an einem Oberflächenschnittpunkt benutzt werden können, solange das zugrunde liegende Objekt untransformiert bleibt. Gerade für geschlossene Polygonnetze entspricht das einer Verteilung der Normalen auf der Oberfläche einer Kugel, weshalb viele unterschiedliche Farben in der Textur zum Vorschein kommen.

Um nun für eine Objektraum-“Normal Map“ die Normale an einem Schnittpunkt zu erhalten, sucht man den zum Schnittpunkt passenden Texel wie im vorherigen Abschnitt beschrieben. Die einzelnen Bytes des im Texel gespeicherten RGB-Werts sind dann r_{RGB} für Rot, g_{RGB} für Grün und b_{RGB} für Blau, wobei jede Komponente maximal den Wert 255 annehmen kann. Die Komponenten $n'.x$, $n'.y$ und $n'.z$ der neuen variierten Normalen $\mathbf{n}'$ ergeben sich dann durch:

$$n'.x = \frac{2 \cdot r_{RGB} - r_{RGB}}{255}; \quad n'.y = \frac{2 \cdot g_{RGB} - g_{RGB}}{255}; \quad n'.z = \frac{2 \cdot b_{RGB} - b_{RGB}}{255}; \quad (9.1)$$

Wird das Objekt durch Transformationsmatrix $\mathbf{T}$ transformiert, so muss die aus der Objektraum-„Normal Map“ erhaltene Normale mit $(\mathbf{T}^{-1})^T$ transformiert werden.

Eine Tangentialraum-“Normal Map“ (Abb. 9.5) bezieht sich hingegen auf die Koordinatenbasis, die durch einen Vektor senkrecht zur Oberfläche und zwei Tangentialvektoren gegeben ist. Der Ursprung der Basis befindet sich am Strahl-Objekt Schnittpunkt. Da auch bei unebenen Oberflächen die meisten Normalen relativ senkrecht auf der Oberfläche stehen und der Blauanteil der Textur für diese senkrechte (z-) Komponente verantwortlich ist, erscheint die Textur überwiegend bläulich.

Die Konstruktion der Koordinatenbasis beginnt mit der Festlegung der z-Achse. Dafür kann die beim Schnitttest berechnete Normale verwendet werden. Es ist jedoch auch möglich, eine interpolierte Normale zu benutzen, wie sie bei der glatten Schattierung verwendet wird,

¹Quelle: <http://docs.cryengine.com/display/SDKDOC4/Tangent+Space+Normal+Mapping>

wodurch man harte Übergänge vermeidet. Prinzipiell gibt es für die x- und y-Achse, die durch Tangentialvektoren festgelegt werden, beliebig viele Orientierungen, solange alle 3 Vektoren orthogonal zueinander stehen. Denn jede Drehung der x- und y-Achse um die z-Achse erzeugt weiterhin eine korrekte Koordinatenbasis im Tangentialraum. Für eine konsistente Schattierung ist es jedoch erforderlich, die Tangentialvektoren entlang der uv-Koordinaten auszurichten. Im Fall von triangulierten Objekten betrachtet man, wie sich die uv-Koordinaten entlang der Dreieckskanten verändern, indem man die Differenzen $\Delta \mathbf{uv}_0 = \mathbf{uv}_1 - \mathbf{uv}_0$ und $\Delta \mathbf{uv}_1 = \mathbf{uv}_2 - \mathbf{uv}_0$ (vgl. Abb.9.1) bildet. Ebenso erzeugt man die Dreieckskanten $\mathbf{e}_0 = \mathbf{p}_1 - \mathbf{p}_0$ und $\mathbf{e}_1 = \mathbf{p}_2 - \mathbf{p}_0$, wobei sich die uv-Koordinate $\mathbf{uv}_0$ am Punkt $\mathbf{p}_0$, $\mathbf{uv}_1$ am Punkt $\mathbf{p}_1$ usw. befindet. Anhand der Komponenten von $\Delta \mathbf{uv}_0$ und $\Delta \mathbf{uv}_1$ wird eine Linearkombination mit den benötigten Basisvektoren d.h. mit der Tangenten $\mathbf{t}$ und mit der Bitangenten $\mathbf{b}$ gesucht, die den Kanten $\mathbf{e}_0$ und $\mathbf{e}_1$ entspricht [ope15]:

$$\begin{aligned}\mathbf{e}_0 &= \Delta \mathbf{uv}_0.u \cdot \mathbf{t} + \Delta \mathbf{uv}_0.v \cdot \mathbf{b} \\ \mathbf{e}_1 &= \Delta \mathbf{uv}_1.u \cdot \mathbf{t} + \Delta \mathbf{uv}_1.v \cdot \mathbf{b}\end{aligned}$$

wobei $\Delta \mathbf{uv}_0.u$ die u -Komponente von $\Delta \mathbf{uv}_0$, $\Delta \mathbf{uv}_0.v$ die v -Komponente von $\Delta \mathbf{uv}_0$ usw. ist. Anhand dieses Gleichungssystems lässt sich nun $\mathbf{t}$ und $\mathbf{b}$ bestimmen, durch:

$$\begin{aligned}r &= (\Delta \mathbf{uv}_0.u \cdot \Delta \mathbf{uv}_1.v - \Delta \mathbf{uv}_0.v \cdot \Delta \mathbf{uv}_1.u)^{-1} \\ \mathbf{t} &= (\mathbf{e}_0 \cdot \Delta \mathbf{uv}_1.v - \mathbf{e}_1 \cdot \Delta \mathbf{uv}_0.v) \cdot r \\ \mathbf{b} &= (\mathbf{e}_1 \cdot \Delta \mathbf{uv}_0.u - \mathbf{e}_0 \cdot \Delta \mathbf{uv}_1.u) \cdot r\end{aligned}$$

Die Vektoren $\mathbf{t}$, $\mathbf{b}$ und $\mathbf{n}$ sind zwar linear unabhängig bilden jedoch keine ONB, weshalb das Gram-Schmidtsches Orthogonalisierungsverfahren eingesetzt wird, um diese Eigenschaft herzustellen. Zu beachten ist, dass dabei die Richtung von $\mathbf{n}$ unverändert bleibt. Im Kontext des Verfahrens von Gram-Schmidt bedeutet das, dass man den Algorithmus mit dem Vektor $\mathbf{n}$ beginnt.

Wie bei der Objektraum-“Normal Map“ wird nun der zum Strahl-Objekt Schnittpunkt passende Texel gesucht und ein Vektor $\mathbf{m}$ gemäß der Gleichungen 9.1 erzeugt. Jedoch ist dies noch nicht die endgültige Normale $\mathbf{n}'$. Diese erhält man durch:

$$\mathbf{n}' = \mathbf{t} \cdot \mathbf{m}.x + \mathbf{b} \cdot \mathbf{m}.y + \mathbf{n} \cdot \mathbf{m}.z$$

wobei $\mathbf{m}.x$, $\mathbf{m}.y$ und $\mathbf{m}.z$ die x, y und z-Komponenten des Vektors $\mathbf{m}$ sind.

10 Bildkorrektur

Ein bekanntes Problem der Computergrafik sind die Alias-Effekte. Zu den bekanntesten gehört die Treppenbildung an nahezu horizontal bzw. vertikal verlaufenden Kanten, oder der Moiré-Effekt bei dem z.B. durch Überlagerung zweier feiner Gitter ein gröberes Muster entsteht. Beispiele hierfür liefert Abb. 10.2. So kann an den horizontal verlaufenden Linien des Schachbretts Treppenbildung beobachtet werden. Ebenso zeigt sich eine Form des Moiré-Effekt und zwar in den hinteren Reihen des Schachbretts, in denen Kacheln wieder größer werden.

Eine Erklärung für diese Erscheinungen findet man in der Signalverarbeitung. Dort tritt „Aliasing“ auf, wenn das ursprüngliche Signal nicht mit einer ausreichend hohen Frequenz abgetastet wird, und deshalb nicht vollständig rekonstruiert werden kann. Um ein Signal fehlerfrei rekonstruieren zu können muss die Abtastfrequenz mindestens doppelt so hoch sein, wie die höchsten Frequenzanteile des Signals. Die Grenzfrequenz ab der eine fehlerfreie Rekonstruktion möglich ist, heißt Nyquist-Frequenz.

Im Kontext des „Raytracing“ ist das Ursprungssignal eine kontinuierliche Funktion über, auf den Sensor einfallender, Strahldichten. Dieses Signal wird an diskreten Punkten abgetastet. Bisher geschah dies, indem Strahlen durch den Mittelpunkt eines PEE geschossen wurden. Zur Bildkorrektur schießt man durch ein einzelnes PEE mehrere Strahlen, um so die Abtastfrequenz zu erhöhen und das Ursprungssignal so gut wie möglich rekonstruieren zu können.

10.1 Erzeugung geeigneter Abtastpunkte

Das Verfahren zur Erzeugung weiterer Abtastpunkte auf den PEE wird auch als „Sampling“ bezeichnet. Bevor die einzelnen „Sampling“-Verfahren vorgestellt werden, werden informell Sachverhalte aus der Signalverarbeitung erläutert, um besser nachvollziehen zu können, was beim „Sampling“ beachtet werden sollte.

Multidimensionale Signale zeigen eine räumliche Änderung, wie z.B. die Verteilung der Strahldichten über die Projektionsebene. Solche Signale lassen sich als Funktionen im Ortsraum darstellen. Durch die Fourier-Transformation lassen sich Signale vom Ortsraum in den Frequenzraum transformieren. Hier wird sichtbar wie viele Anteile des Signals in einem bestimmten Frequenzband liegen. Das Signal dargestellt als Funktion wird abgetastet, indem die Funktion mit einem Dirac-Kamm multipliziert wird. Der Dirac-Kamm besteht aus einer Folge von Dirac-Stößen, die sich nach konstanten Perioden T der Abtastfrequenz wiederholen. Die Periode T hängt von der Anzahl der Abtastpunkten - den „Samples“ - ab. Je mehr Abtastpunkte vorhanden sind desto kleiner ist T . Dieser Prozess der Signalabtastung kann auch im Frequenzbereich beobachtet werden. Nur handelt es sich hier um eine Faltung

des Frequenzspektrums mit einem Dirac-Kamm der Periode $\frac{1}{T}$. Sind nicht genug Abtastpunkte vorhanden, überlagern sich Frequenzbereiche des Signals. Diese Überlagerungen im Frequenzraum lassen sich nicht mehr fehlerfrei rekonstruieren.

Zwei Dinge versucht man deshalb bei den „Sampling“-Verfahren zu erreichen. Einerseits kann das Überlappen von Frequenzbereichen im Frequenzraum reduziert werden, indem man mehr Strahlen durch ein PEE schießt und so die Anzahl an Abtastpunkten erhöht, denn dies vergrößert die Abstände der Dirac-Stöße im Frequenzraum. Dennoch ist es möglich, dass sich Überlappungen nicht komplett verhindern lassen, weil dazu zu viele „Samples“ benötigt werden, was sich linear in der Laufzeit niederschlägt. In diesem Fall versucht man die Regelmäßigkeit, mit der Abtastpunkt auf Abtastpunkt folgt, zu stören. Denn diese Regelmäßigkeit verursacht die störenden Alias-Effekte und Bildartefakte. Ziel ist es daher das PEE möglichst mit geringer Diskrepanz (das heißt die „Samples“ sollten einen möglichst großen Abstand zueinander haben) jedoch nicht regelmäßig durch „Samples“ aufzuteilen. Dadurch werden Alias-Effekte und Bildartefakte durch Rauschen ersetzt, was für den Betrachter natürlicher wirkt.

- Mittelpunkt „Sampling“: Die einfachste Art des „Sampling“, indem durch den Mittelpunkt eines PEE ein Primärstrahl geschossen wird. Abb. 10.2 zeigt durch die geringe und regelmäßige Abtastung Treppenbildung an Kanten und Schatten des Buddhas, sowie horizontal und vertikal verlaufenden Kanten der Kacheln. Nahe des Horizonts kommt es zu Bildartefakte durch wieder größer werdende Kacheln.
- „Random Sampling“: Die Komponenten eines „Samples“ werden gleichverteilt entlang der x- und y-Achse des PEE gezogen. Abb. 10.3 zeigt den Ausschnitt mit 4 „Samples“. Durch das komplett zufällige ziehen der „Samples“, kann es zu schlechten Verteilungen durch Pulkbildung kommen. Zwar wurde die Treppenbildung entfernt, dafür erscheinen Kanten und Schatten ausgefranst. Die hinteren Bildartefakte wurden durch Rauschen ersetzt.
- „Stratified Sampling“: Das PEE wird in kleinere gleichgroße Quadrate unterteilt. Innerhalb dieser Quadrate wird je ein „Sample“ durch „Random Sampling“ erzeugt. Abb. 10.4 zeigt den Ausschnitt mit 4 „Samples“. Der Ausschnitt ähnelt sehr dem des „Random Sampling“, jedoch mit weniger Rauschen, da durch die weitere Aufteilung des PEE die Pulkbildung verhindert wird.
- „Hammersley Sampling“: Aus n zu erzeugenden „Samples“ berechnet sich die erste Komponente des i -ten „Samples“ aus $\frac{i}{n}$ und die zweite aus dem i -ten Element der „Van der Corput“ Zahlenreihe zur Basis 2. Abb. 10.5 zeigt bei 4 „Samples“ bessere Glättung der Kanten als „Random“ oder „Stratified“ Sampling, da kein Ausfransen zu erkennen ist. Es zeichnet sich durch geringe Diskrepanz und unregelmäßiger Verteilung der „Samples“ aus. Jedoch fehlt der Zufallsfaktor bei diesem Verfahren, was neue Alias-Effekte anstelle von Rauschen ins Bild einbringt. Dies wird durch 2 graue Streifen in der Nähe des Horizonts deutlich.

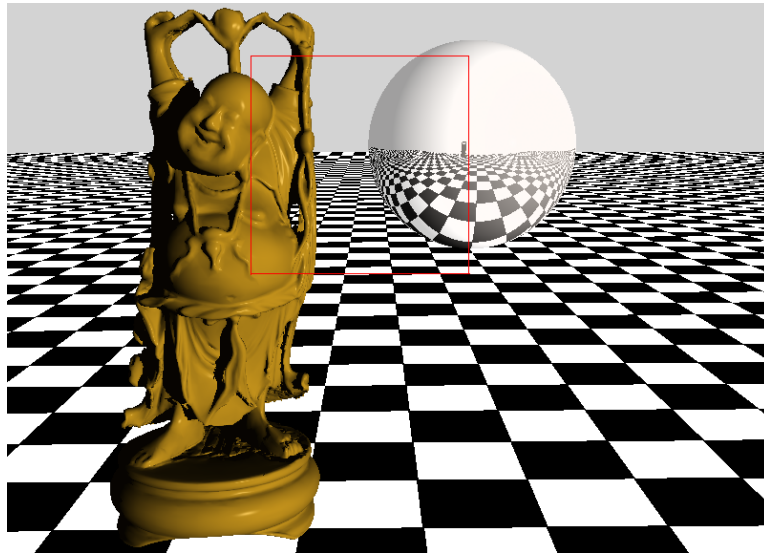


Abb. 10.1: Komplette Szene „gerendert“ mit Mittelpunkt „Sampling“. Die folgenden Ausschnitte verwenden unterschiedliche „Sampling“ Verfahren. Die Zahl steht für die Anzahl verwendeter „Samples“.

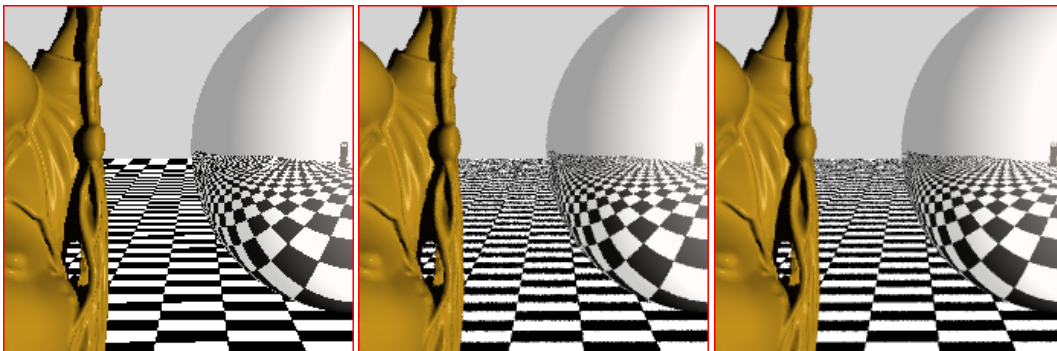


Abb. 10.2: 1 Mittelpunkt

Abb. 10.3: 4 „Random“

Abb. 10.4: 4 „Stratified“

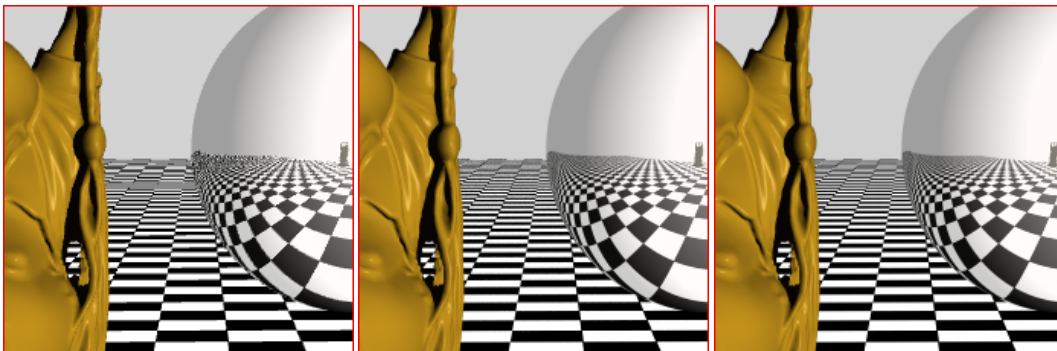


Abb. 10.5: 4 „Hammersley“

Abb. 10.6: 64 „Stratified“

Abb. 10.7: 64 „Hammersley“

11 Galerie

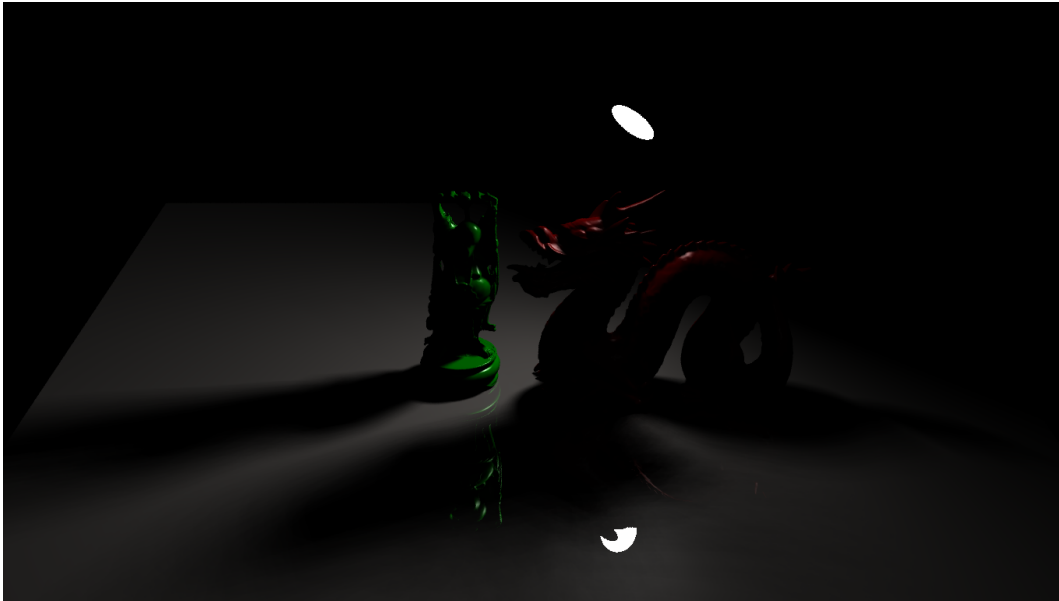


Abb. 11.1: Phong schattierter „Stanford Buddha“ und „Stanford Dragon“ auf spiegelndem Untergrund. Die Szene wird von zwei runden Flächenlichter beleuchtet.

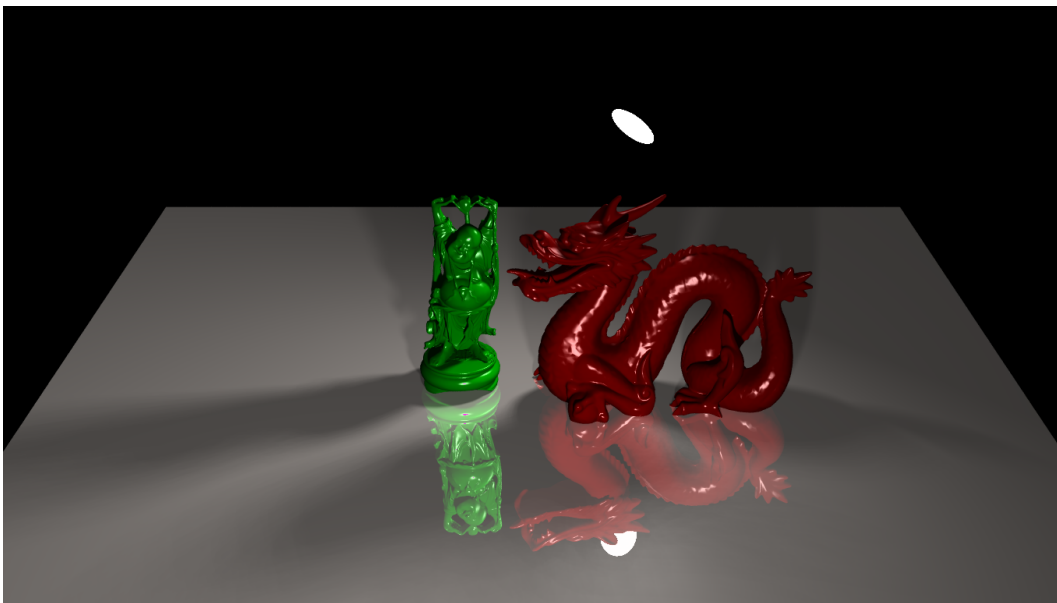


Abb. 11.2: Phong schattierter „Stanford Buddha“ und „Stanford Dragon“ auf spiegelndem Untergrund. Die Szene wird von zwei runden Flächenlichter beleuchtet.



Abb. 11.3: Texturiertes Modell eines T-Rex, bestehend aus Bildtextur und „Normal Map“.

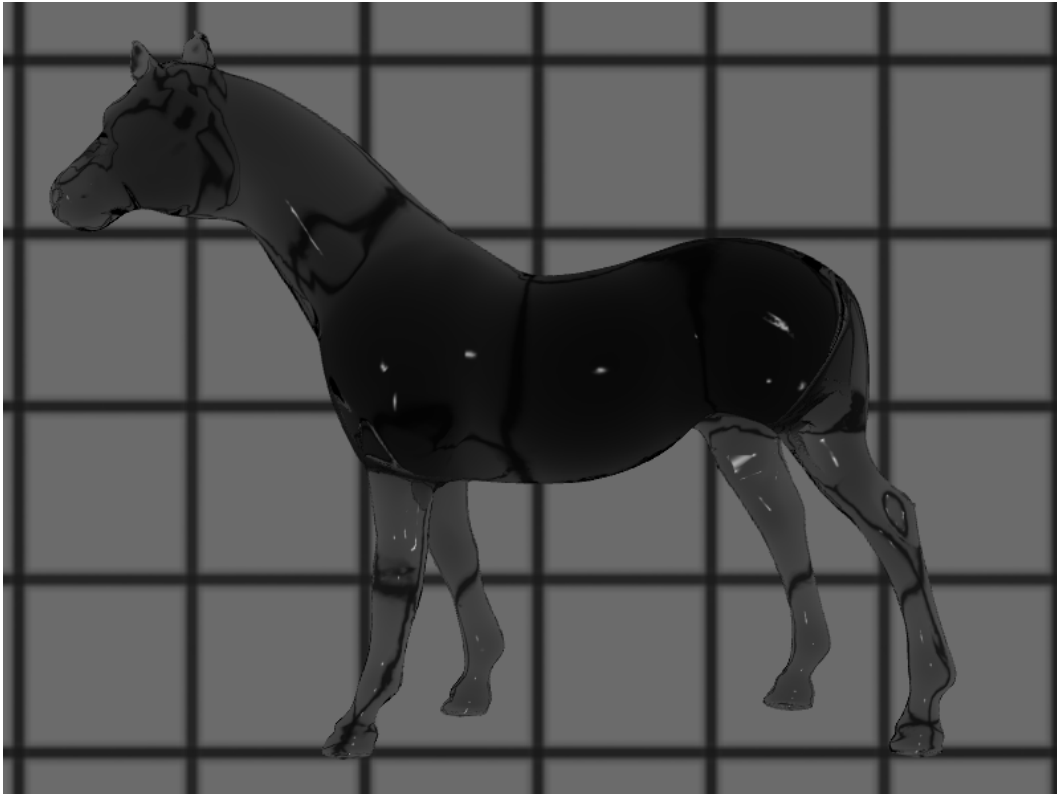


Abb. 11.4: Pferdmodell gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,3 und direktionalem Licht. Das Licht wird im Bauch des Pferdes stärker absorbiert als in den dünnen Beinen, wodurch diese dunkler wirkt.

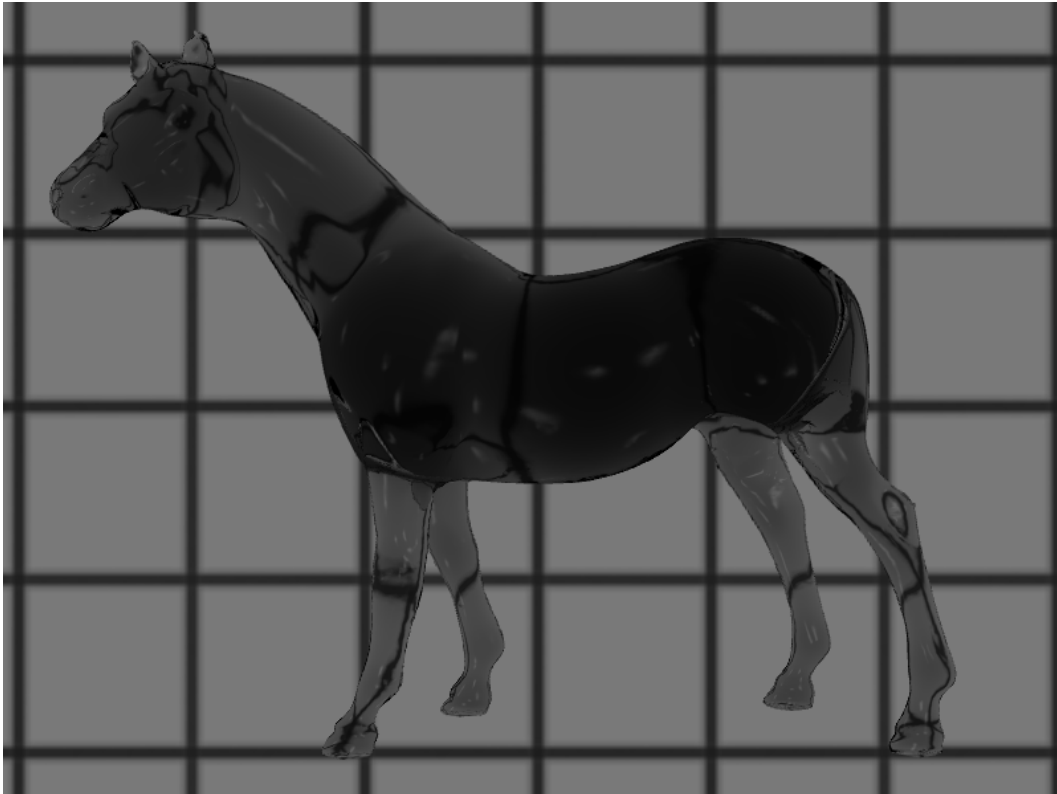


Abb. 11.5: Pferdmodell gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,3 und Flächenlicht. Das Licht wird im Bauch des Pferdes stärker absorbiert als in den dünnen Beinen, wodurch diese dunkler wirkt.

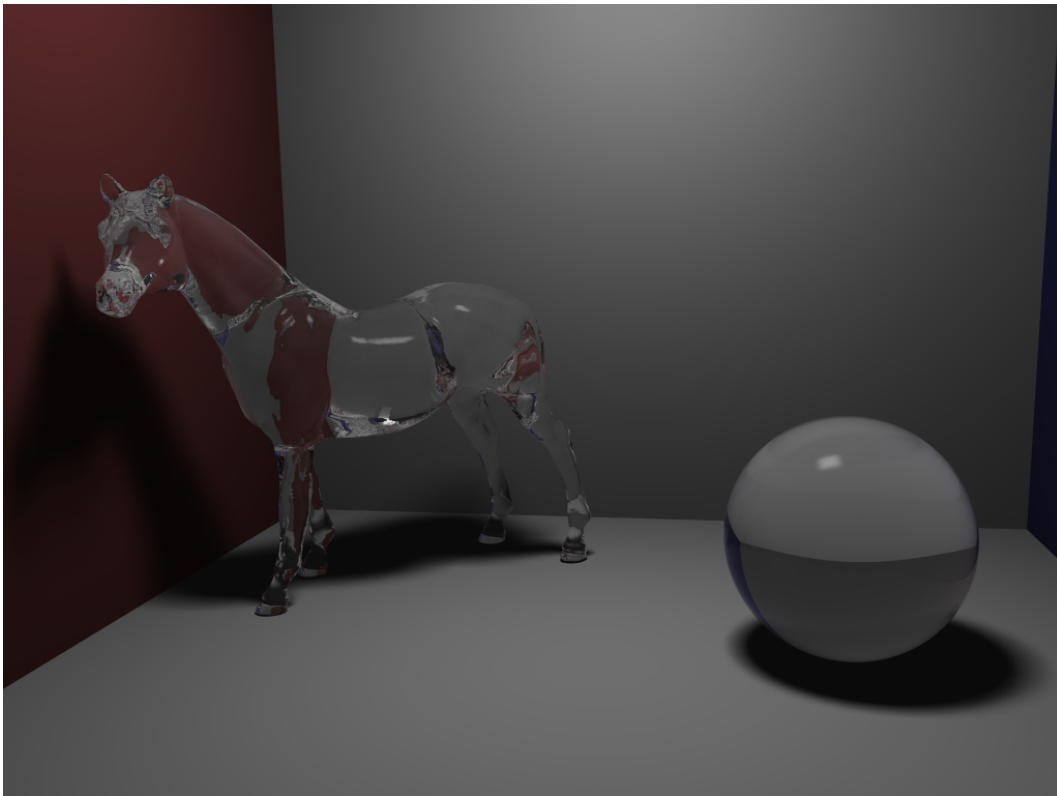


Abb. 11.6: Pferdmodell und Kugel in „Cornell box“. Gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,35 und Flächenlicht.

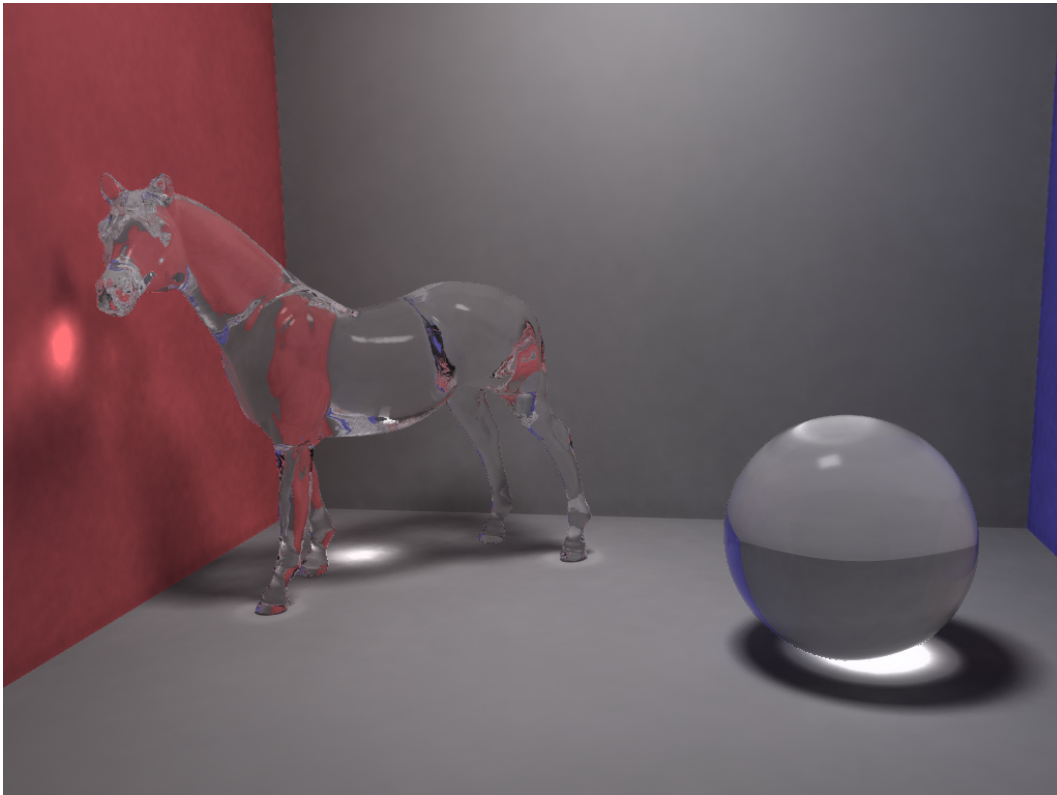


Abb. 11.7: Pferdmodell und Kugel in „Cornell box“. Gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,35 und Flächenlicht. Verwendung von „Photon Mapping“ mit 5 Millionen Photonen, zur Darstellung von Kaustiken und indirekter Beleuchtung. Zur Berechnung der Bestrahlungsstärke wurde eine konvexe Hülle verwendet, um dunkle Kanten und Ecken zu vermeiden.

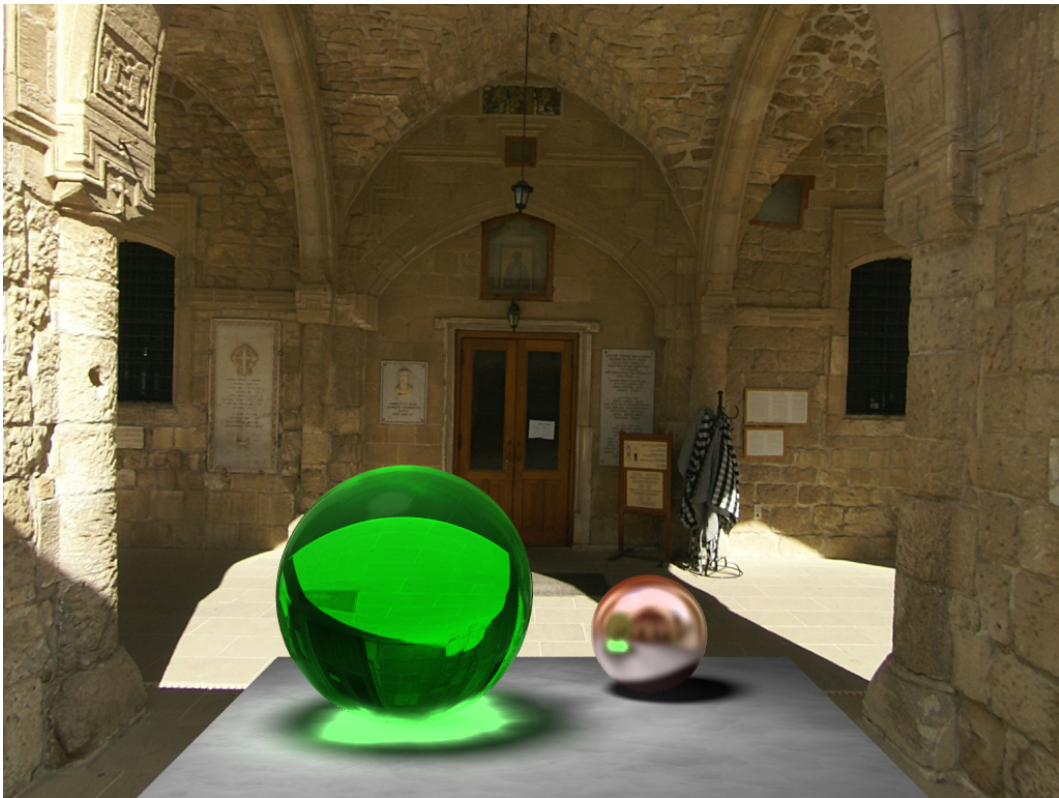


Abb. 11.8: Transparente und matt spiegelnde Kugel in einer Skybox der Lazarus-Kirche.
„Photon Mapping“ erzeugt Kaustiken.

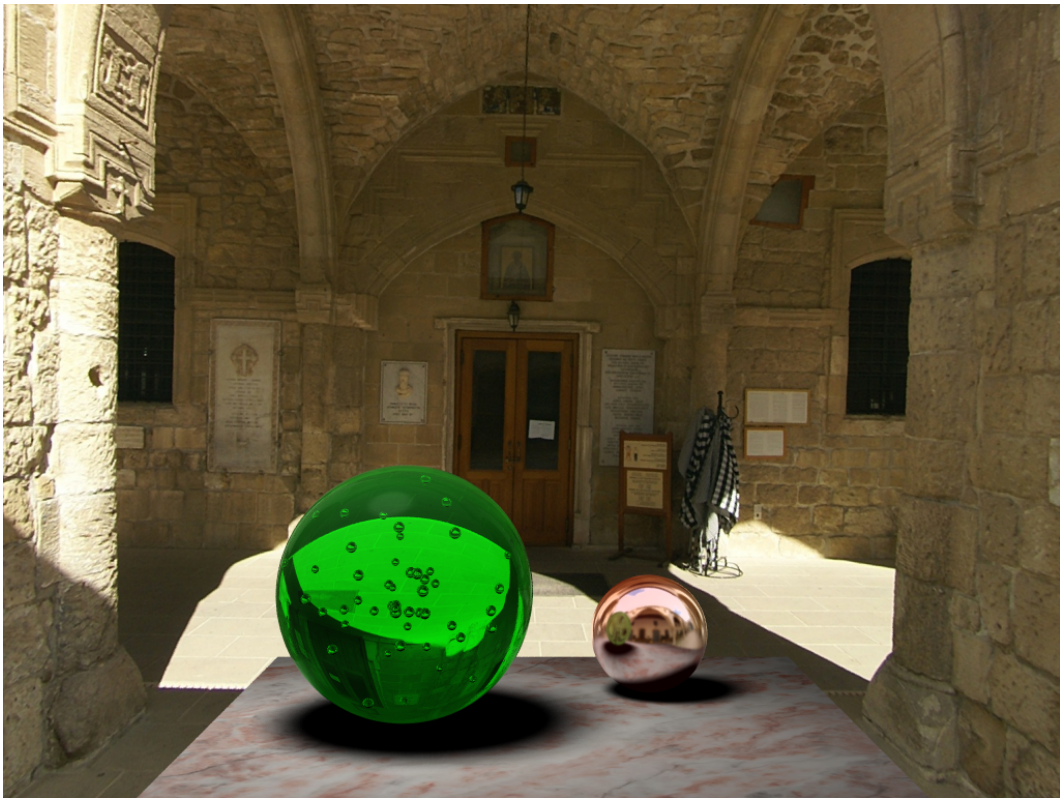


Abb. 11.9: Transparente Kugel mit eingeschlossen Luftblasen und spiegelnde Kugel in einer Skybox der Lazarus-Kirche.

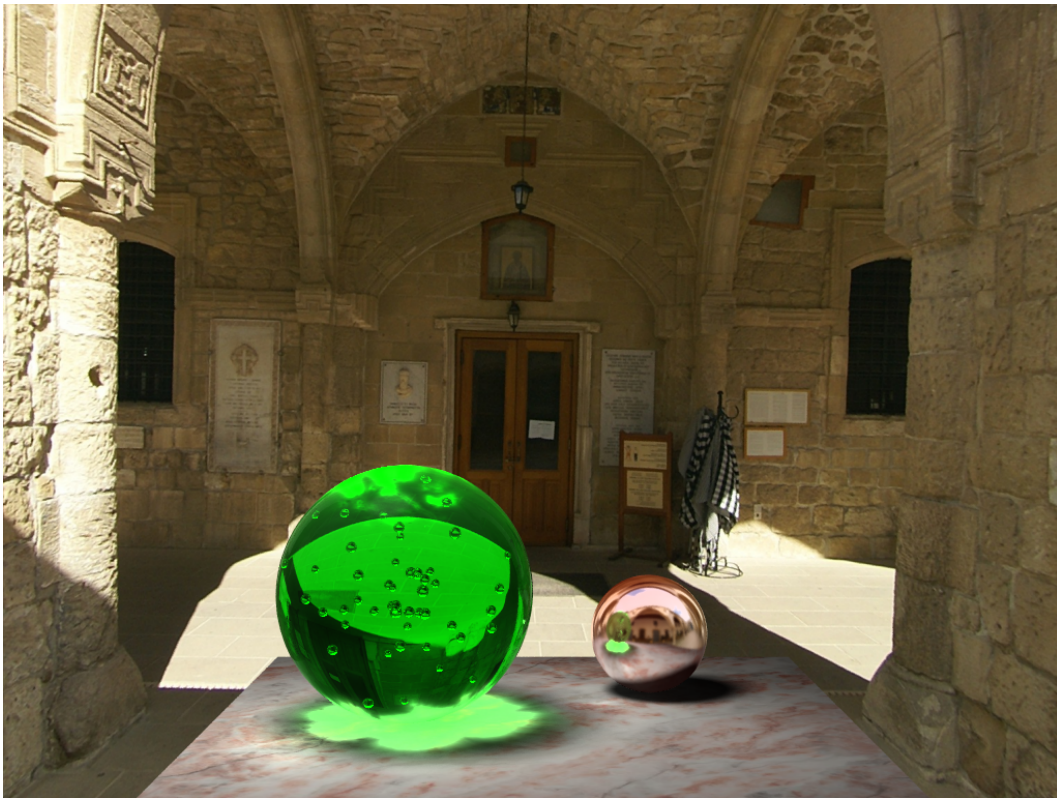


Abb. 11.10: Transparente Kugel mit eingeschlossen Luftblasen und spiegelnde Kugel in einer Skybox der Lazarus-Kirche. „Photon Mapping“ sorgt für Kaustiken.

Literaturverzeichnis

- [AD14] ARANSAY, Jesús ; DIVASÓN, Jose: Applications of the Gauss-Jordan algorithm, done right. In: *CoRR* abs/1401.5910 (2014). <http://dblp.uni-trier.de/db/journals/corr/corr1401.html#AransayD14>
- [Mat10] MATT PHARR, Greg H.: *Physically Based Rendering, Second Edition: From Theory To Implementation*. Morgan Kaufmann, 2010
- [MT97] MÖLLER, Tomas ; TRUMBORE, Ben: Fast, Minimum Storage Ray-Triangle Intersection. In: *J. Graphics, GPU, & Game Tools* 2 (1997), Nr. 1, 21–28. <http://dblp.uni-trier.de/db/journals/jgtools/jgtools2.html#MollerT97>
- [Mö27] MÖBIUS, August F.: *Der Barycentrische Calcul : ein neues Hülfsmittel zur analytischen Behandlung der Geometrie*. 1827 <http://www.bibsonomy.org/bibtex/2d9e09cdc2c1a338418645991632df3b5/guillem.palou>. – ISBN 1144890934
- [ope15] *OpenGL Normal Mapping*. <http://www.opengl-tutorial.org/intermediate-tutorials/tutorial-13-normal-mapping>. Version: Juli 2015
- [Red15] REDDY, Martin: *Spezifikation OBJ-Format*. <http://www.martinreddy.net/gfx/3d/OBJ.spec>. Version: Juni 2015
- [Suf07] SUFFERN, Kevin: *Ray Tracing from the Ground Up*. Taylor & Francis, 2007 (Ak Peters Series). <http://books.google.de/books?id=9jJHAQAAIAAJ>. – ISBN 9781568812724
- [WK06] WÄCHTER, Carsten ; KELLER, Alexander: Instant Ray Tracing: The Bounding Interval Hierarchy. In: AKENINE-MÖLLER, Tomas (Hrsg.) ; HEIDRICH, Wolfgang (Hrsg.): *Rendering Techniques*, Eurographics Association, 2006. – ISBN 3-905673-35-5, 139–149

Abbildungsverzeichnis

3.1	Orthografische Kamera	8
3.2	Lochkamera	8
3.3	Die Projektionsebene mit Höhe h , Breite b , Mittelpunkt $\mathbf{p}_{proj}$, „Up-Vektor“ $\mathbf{u}_{proj}$ und Normale $\mathbf{n}_{proj}$. Sie ist unterteilt in gleichgroße Quadrate (PEE) mit Kantenlänge l_{PEE}	9
4.1	Schnitt einer Kugel mit Ebene, Scheibe, Rechteck und Dreieck. Der zum Betrachter nächste Schnittpunkt wird zu korrekten Darstellung der Objektdurchdringungen benötigt.	12
4.2	Definition eines Rechtecks durch linken unteren Stützpunkt $\mathbf{a}$, Kantenvektoren $\mathbf{e}_1, \mathbf{e}_2$ und Normale $\mathbf{n}$. Ein Strahl schneidet das Rechteck am Schnittpunkt $\mathbf{r}$	13
4.3	Baryzentrische Koordinaten im Dreieck. Alle Punkte im Dreieck können als gewichtete Summe der 3 Eckpunkten dargestellt werden.	15
5.1	Die Raute und ihre Normale $\mathbf{n}$ werden auf die gleiche Art und Weise transformiert und zwar auf die doppelte Breite und halbe Höhe skaliert. Nach der Skalierung steht $\mathbf{n}$ nicht mehr orthogonal auf der Oberfläche. Die korrekte Normale ist $\mathbf{n}'$. D.h. die Normale muss bei der Transformation gesondert behandelt werden.	20
6.1	Pferdemodell und Kugel in einer Box	23
6.2	Pferd und Kugel mit spekulärer Oberfläche. Die Größen der Glanzpunkte ergeben sich durch den Exponenten $n = 30$	24
6.3	Pferd und Kugel mit spekulärer Oberfläche. Die Größen der Glanzpunkte ergeben sich durch den Exponenten $n = 100$	24
6.4	Das Aussehen der matt spiegelnden Oberflächen wird erzeugt, indem Strahlen konisch um die perfekte Reflexionsrichtung $\mathbf{r}$ in die Szene geschickt werden.	25
6.5	Das Aussehen der perfekt spiegelnden Oberflächen wird erzeugt, indem Strahlen entlang der perfekten Reflexionsrichtung $\mathbf{r}$ in die Szene geschickt werden.	25
6.6	Dielektrika mit einem Brechungsindex von 1,3. Dies entspricht näherungsweise dem Übergang von Luft zu Eis	25
6.7	Dielektrika mit einem Brechungsindex von 0,75. Dies entspricht näherungsweise dem Übergang von Wasser zu Luft	25
7.1	Verwendung nur einer Normalen pro Dreieck führt zu facettierter Schattierung	28
7.2	Einbeziehung und Mittelwertbildung benachbarter Dreiecksnormalen sorgt für glattere Schattierung	28

Abbildungsverzeichnis

8.1	a) Rekursive Unterteilung der Szene anhand von Trennebenenkandidaten. Die jeweils längste Achse wird mittig durch einen Trennebenenkandidat geteilt. b) Tatsächliche Aufteilung des Raums nach Ablegen der Objekte in Blättern. Leere Räume benötigen keine weitere Unterteilung. c) Rote Klammern zeigen die maximale Ausdehnung der Objekte des linken Kindknoten entlang der geteilten Achse. Blaue Klammern zeigen minimale Ausdehnung der Objekte des rechten Kindknoten. ¹	32
8.2	Die Testszene „Kugeln“ zeigt 1 Million, innerhalb eines Würfels mit der Kantenlänge 9, gleichverteilte Kugeln. Die Auflösung des Bilds beträgt 800×800 Pixel.	34
8.3	Die Testszene „Buddhas“ zeigt 1000, innerhalb einer Kugel mit Radius 9, gleichverteilte Stanford Buddhas. Die Auflösung des Bilds beträgt 800×800 Pixel.	34
8.4	Zeitmessung in Sekunden der Testszene „Kugeln“. Das Diagramm ist unterteilt in Segmente, zur Zeitmessung der Konstruktion sowie der Traversierung der BIH. Die Konstruktion findet immer auf einem CPU Kern statt, wohingegen die Traversierung mit 1, 2 und 4 Kernen durchgeführt wird. Die Balken innerhalb eines Segments stehen für die Anzahl der Objekte in der Szene und zeigen an, ob die BIH verwendet wurde oder nicht („Ohne“).	35
8.5	Zeitmessung in Sekunden der Testszene „Buddhas“. Das Diagramm ist unterteilt in Segmente, zur Zeitmessung der Konstruktion sowie der Traversierung der BIH. Die Konstruktion findet immer auf einem CPU Kern statt, wohingegen die Traversierung mit 1, 2 und 4 Kernen durchgeführt wird. Die Balken innerhalb eines Segments stehen für die Anzahl der Objekte in der Szene. Zum Rendern der Buddhas wurde Instanziierung verwendet, wodurch die Konstruktionszeit gleich bleibt.	36
9.1	Textur als 2D-Array mit uv-Koordinaten als Spaltenvektor der Form $\begin{pmatrix} u \\ v \end{pmatrix} \in \mathbb{R}^2$	37
9.2	Das Modell eines T-Rex ohne „Normal Mapping“: Laufzeit 2,08 Sekunden	38
9.3	Das selbe Modell mit „Normal Mapping“: Laufzeit 2,1 Sekunden	38
9.4	Objektraum-„Normal Map“ ²	39
9.5	Tangentialraum-„Normal Map“ ³	39
10.1	Komplette Szene „gerendert“ mit Mittelpunkt „Sampling“. Die folgenden Ausschnitte verwenden unterschiedliche „Sampling“ Verfahren. Die Zahl steht für die Anzahl verwendeter „Samples“.	43
10.2	1 Mittelpunkt	43
10.3	4 „Random“	43
10.4	4 „Stratified“	43
10.5	4 „Hammersley“	43
10.6	64 „Stratified“	43
10.7	64 „Hammersley“	43
11.1	Phong schattierter „Stanford Buddha“ und „Stanford Dragon“ auf spiegelndem Untergrund. Die Szene wird von zwei runden Flächenlichter beleuchtet.	44
11.2	Phong schattierter „Stanford Buddha“ und „Stanford Dragon“ auf spiegelndem Untergrund. Die Szene wird von zwei runden Flächenlichter beleuchtet.	45
11.3	Texturiertes Modell eines T-Rex, bestehend aus Bildtextur und „Normal Map“.	46

Abbildungsverzeichnis

11.4 Pferdmodell gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,3 und directionalem Licht. Das Licht wird im Bauch des Pferdes stärker absorbiert als in den dünnen Beinen, wodurch diese dunkler wirkt.	47
11.5 Pferdmodell gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,3 und Flächenlicht. Das Licht wird im Bauch des Pferdes stärker absorbiert als in den dünnen Beinen, wodurch diese dunkler wirkt.	48
11.6 Pferdmodell und Kugel in „Cornell box“. Gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,35 und Flächenlicht.	49
11.7 Pferdmodell und Kugel in „Cornell box“. Gerendert mit dielektrischem Material unter Verwendung eines Brechungsindex von 1,35 und Flächenlicht. Verwendung von „Photon Mapping“ mit 5 Millionen Photonen, zur Darstellung von Kaustiken und indirekter Beleuchtung. Zur Berechnung der Bestrahlungsstärke wurde eine konvexe Hülle verwendet, um dunkle Kanten und Ecken zu vermeiden.	50
11.8 Transparente und matt spiegelnde Kugel in einer Skybox der Lazarus-Kirche. „Photon Mapping“ erzeugt Kaustiken.	51
11.9 Transparente Kugel mit eingeschlossen Luftblasen und spiegelnde Kugel in einer Skybox der Lazarus-Kirche.	52
11.10 Transparente Kugel mit eingeschlossen Luftblasen und spiegelnde Kugel in einer Skybox der Lazarus-Kirche. „Photon Mapping“ sorgt für Kaustiken.	53